

DORA: A Scalable Asynchronous Reinforcement Learning System for Language Model Training

Tianhao Hu,[†] Xiangcheng Liu,[†] Yuchun Miao,[†] Youshao Xiao,[†] Hongyu Zang, Yang Zheng
Xuan Huang, Jinrui Ding, Yufei Zhang, Yu Yang, Yi-Kai Zhang, Yueqing Sun
Chengcheng Han, Xiandi Ma,^{*} Wei Wang, Qi Gu,^{*} Yerui Sun, Yuchen Xie, Xunliang Cai
Meituan Longcat Team
maxiandi02@meituan.com, guqi03@meituan.com

Abstract

Reinforcement learning (RL) has become a critical paradigm for large-scale post-training of LLMs in industrial settings, yet it faces a structural *long-tail dilemma*: rollout efficiency is bottlenecked by the longest trajectories, which are often the most valuable for RL training. Existing approaches alleviate this dilemma at the cost of either system overhead (e.g., re-prefill in partial-rollout methods) or algorithmic compromises (e.g., discarded long trajectories in replication-based methods). Both approaches implicitly assume that all rollout instances must synchronize around a single policy version, creating an unavoidable batch barrier. We propose **DORA** (**D**ynamic **O**Rchestration for **A**synchronous **R**ollout), which breaks this assumption by maintaining multiple policy versions concurrently within the rollout cluster to solve the skewed generation problem without algorithmic compromise. DORA combines three mechanisms: *multi-version streaming training* that decouples trajectory completion from batch barrier, a centralized *load-balancing orchestrator* that re-partitions resources across versions, and nearly *zero-re-prefill migration* that transfers KV-Cache directly across same-version instances. Experiments on open-source benchmarks show that DORA achieves up to $2.12\times$ end-to-end throughput improvement and $8.2\times$ rollout-stage acceleration over synchronous training while preserving convergence parity. **In real-world production deployment** with thousands of accelerators, DORA achieves up to $6.2\times$ rollout speedup, successfully training a $\sim 500\text{B}$ -parameter MoE model (i.e., **LongCat-Flash-Thinking**) competitive with state-of-the-art open-source LLMs.

1 Introduction

Reinforcement Learning (RL) has become a pivotal paradigm for LLM post-training, leveraging test-time scaling [35] to advance complex reasoning and agentic capabilities [3, 23, 31, 37, 39]. The RL training loop sequentially cycles through rollout (trajectory generation), experience preparation (reward and reference computation), and model training. Among these stages, rollout accounts for 50%–80% of the total step duration and represents the fundamental training bottleneck [38, 41, 42]. As industrial deployments scale to thousands of accelerators, optimizing rollout efficiency directly determines the overall training cost and turnaround time.

This rollout bottleneck is fundamentally driven by a *long-tail dilemma* inherent to complex reasoning tasks: a direct conflict between algorithmic value and hardware efficiency. In mathematics and coding

[†]Equal contribution.

^{*}Corresponding authors.

domains, response lengths follow a highly skewed distribution where the 99th-percentile output can exceed the median by over an order of magnitude (Figures 1 and 2). The dilemma arises because these exceptionally long trajectories carry the highest information density—encapsulating the intricate chain-of-thought (CoT) reasoning steps that are the primary source of emergent capabilities [23, 11]—making them indispensable for RL training. However, because decoding is inherently memory-bound, we cannot simply accelerate them by scaling compute, nor can we discard them without catastrophic algorithmic degradation. Consequently, under standard synchronous training, the entire batch is held hostage by a small fraction of these most valuable, yet longest trajectories, leaving the majority of devices completely idle.

To address this dilemma, asynchronous training has been proposed, with current efforts primarily focusing on two directions. *Replication-based methods* [9, 45] shorten rollout duration by oversampling and dropping the in-flight long trajectories once enough complete. This *discards* precisely the chain-of-thought trajectories highlighted above, and the resulting *length-biased* distribution distorts the advantage estimation in group-relative methods such as GRPO [30]. *Partial-rollout methods* [36, 41, 8, 48] segment long trajectories at each weight update and resume them under the new policy. On the system side, every update invalidates the KV-Cache and forces a *full re-prefill* that grows dramatically with context length (Figure 3) and is further amplified in MoE architectures. On the algorithmic side, a trajectory now stitches multiple policy versions, potentially downgrades the rollout quality, and departs from the standard RL formulation, risking model performance degradation. Across both directions, existing approaches **alleviate the long-tail dilemma, but at the cost of either additional system overhead or algorithmic compromises.**

We argue that these tradeoffs stem from a common implicit assumption—*single-version rollout*, i.e., the rollout instances only serve a single policy version. Under this assumption, in-flight long-tailed trajectories must either complete before the next policy update or be sacrificed at the update—leaving no room for solutions that avoid both system overhead and algorithmic compromises.

In this paper, we depart from this regime through **DORA (Dynamic ORchestration for Asynchronous Rollout)**, which embodies a *multi-version rollout* paradigm where multiple policy versions coexist within the rollout instances, **resolving the long-tail dilemma without incurring significant system overhead or algorithmic compromises.** At its core, each trajectory is generated entirely under the policy version active at its dispatch, so that long-tailed trajectories run to completion under their original version while new requests proceed under the latest version.

Realizing this design, however, raises three system-level challenges, each addressed by a corresponding mechanism. **Version coexistence.** Maintaining multiple policy versions concurrently requires breaking the synchronous batch barrier so that completed trajectories can flow into training without waiting for the slowest ones. DORA achieves this through *multi-version streaming training*, which dispatches and collects trajectories at the granularity of individual requests across versions, with a sliding window that bounds policy staleness. **Resource fragmentation.** As trajectories of older versions complete, their rollout instances become progressively underutilized while the latest version is over-subscribed. DORA addresses this through a centralized *load-balancing orchestrator*, which continuously re-partitions data-parallel groups across versions in proportion to their pending workloads and migrates requests to rebalance the cluster. **Migration overhead.** Naively migrating a request across instances would re-trigger the prefill phase, which is costly in long-context and MoE settings. DORA avoids this entirely: since the trajectory is generated under the consistent policy version across multiple RL steps, its KV-Cache states are mathematically equivalent across any instance hosting that version, enabling nearly *zero-re-prefill migration* via direct cross-instance KV-Cache transfer. Together, these mechanisms enable DORA to deliver **substantial efficiency gains while maintaining standard RL convergence.** Our main contributions are summarized as follows:

- **Multi-Version Streaming Training.** We propose *multi-version streaming training*, a new asynchronous paradigm that maintains multiple policy versions concurrently within the rollout instance, eliminating the long-tail bubble without introducing significant system overhead or algorithmic compromises.
- **Dynamic Orchestration.** We design a centralized *load-balancing orchestrator* that dynamically re-partitions data-parallel groups across versions in proportion to their pending workloads and migrates requests to rebalance the cluster, eliminating resource fragmentation across coexisting policy versions.

- **Nearly Zero Re-prefill KV-Cache Reuse.** We design *zero re-prefill migration* that transfers KV-Cache directly across same-version instances with negligible communication costs, eliminating prefill recomputation during request migration—especially beneficial for long-context reasoning and agentic scenarios.

- **Extensive Evaluation and Real-World Deployment.** On open-source benchmarks, DORA achieves up to $2.12\times$ end-to-end and $8.2\times$ rollout-stage speedup over synchronous training while preserving model convergence. *Production deployment* with thousands of accelerators further yields up to $6.2\times$ rollout speedup, training a $\sim 500\text{B}$ -parameter MoE model competitive with state-of-the-art open-source LLMs.

2 Preliminaries

2.1 Asynchronous RL Training

RL post-training for LLMs proceeds in iterative steps, each consisting of three stages: *rollout* (sampling responses from the current policy), *experience preparation* (computing rewards and references), and *model training*. In synchronous training, a step cannot begin until all trajectories of the previous step are complete, enforcing a strict batch barrier between rollout and training. To overlap rollout and training, asynchronous methods relax this barrier and allow the training samples to mix trajectories generated under different behavior policy versions.

We take GRPO [30], a variant of PPO, as a representative algorithm. Given a prompt x , G trajectories $\{y_i\}_{i=1}^G$ are sampled per prompt and used to update the policy π_θ via a clipped importance-weighted objective with group-relative advantages $\hat{A}_i$ shared across all tokens of trajectory y_i [11, 44]. Let $v(\cdot)$ denote the version index of a policy and K a configurable upper bound on staleness. For trajectory y_i under behavior policy π_{w_i} updating training policy π_θ , asynchronous training requires:

$$v(\theta) - v(w_i) \leq K, \quad (1)$$

which is the standard condition for convergence guarantees in asynchronous optimization [20, 46]. The asynchronous GRPO objective replaces the single behavior policy with the per-trajectory π_{w_i} in the importance ratio:

$$\mathcal{J}_{\text{async}}(\theta) = \mathbb{E} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{L_i} \sum_{t=1}^{L_i} \min \left(r_{i,t}(\theta) \hat{A}_{i,t}, \text{clip}_\varepsilon(r_{i,t}(\theta)) \hat{A}_{i,t} \right) \right], \quad r_{i,t}(\theta) = \frac{\pi_\theta(y_{i,t} | \cdot)}{\pi_{w_i}(y_{i,t} | \cdot)}, \quad (2)$$

subject to Equation 1. Throughout the paper, we denote the rollout batch size (number of prompts dispatched per step) as RBS and the training batch size (number of trajectories consumed by the training stage) as TBS .

2.2 Long-Tail Dilemma

The core objective of asynchronous RL is to resolve the massive hardware idleness caused by long-tailed generation, which manifests at the system level through two compounding factors:

Workload skewness. In long-context reasoning workloads, response lengths L_i follow a heavily long-tailed distribution. As shown in Figures 1 and 2, the 99th-percentile output exceeds the median by over an order of magnitude on both an open-source benchmark and a production workload. Because decode is memory-bound, this tail cannot be flattened by adding compute. Prefill cost compounds this issue: as shown in Figure 3, prefill duration grows dramatically with input length, making any technique that re-triggers prefill mid-step (e.g., re-prefill after weight updates) increasingly expensive in long-context regimes.

Hardware-level idleness. The rollout phase processes RBS prompts per step, where each request incurs a compute-bound prefill and a memory-bound decode, with concurrency capped by accelerator memory occupied by model weights and KV-Cache. The objective is to minimize total step duration:

$$\min T_{\text{train}} + T_{\text{Prefill}} + T_{\text{Decode}} \Rightarrow \min T_{\text{train}} + T_{\text{Prefill}} + \underbrace{\tau \max_j \max_{i \in \text{Device}_j} \{L_i\}}_{\text{intra-node bubble}} \quad (3)$$

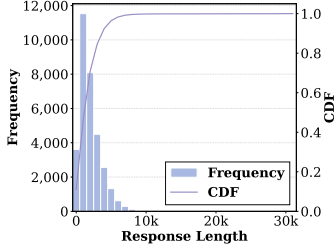


Figure 1: Response length distribution on the DAPO-Math-17K dataset.

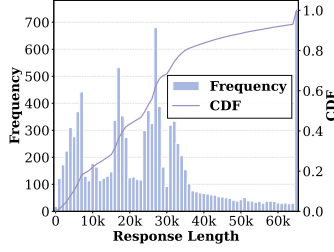


Figure 2: Response length distribution in production. Stacked bar in the end of x-axis reflects overlong truncation.

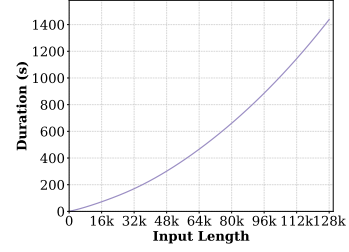


Figure 3: Prefill duration of a ~ 500 B MoE model with expert parallelism size 128 on non-CUDA accelerators.

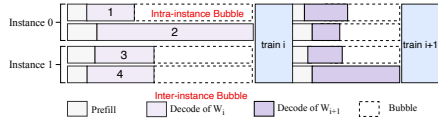


Figure 4: Skewed bubbles in synchronous training: intra-node bubble (idle slots within a device) and inter-node bubble (faster instances waiting for the slowest).

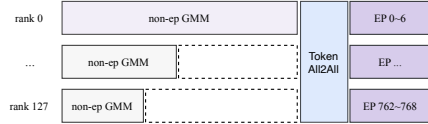


Figure 5: Non-EP GMM workloads become unbalanced under long-tailed inputs (expert parallelism size 128).

where τ is the time per output token (TPOT, approximately constant under fixed batch size for rollout engine). Combined with the long-tailed distribution above, this maxmax structure in the rollout creates two forms of device idleness illustrated in Figure 4: an *intra-node bubble*, where completed slots on a device sit idle while a long-tailed request continues without device saturation, and an *inter-node bubble*, where faster instances wait for the slowest one. In MoE architectures, this skew further propagates into non-EP layers, where the slowest rank stalls the entire EP group (Figure 5).

While long-tail trajectories bottleneck rollout efficiency, their rich learning signals make them indispensable for RL. This *long-tail dilemma* exposes a fundamental conflict between hardware utilization and algorithmic integrity. Consequently, existing asynchronous methods cannot mitigate rollout bubbles without incurring significant re-prefill overhead or algorithmic sampling bias (Section 1, Appendix A). The remainder of this paper introduces a system architecture that natively resolves this tension, eliminating bubbles without incurring significant system overhead or sacrificing algorithmic fidelity.

3 DORA Design

3.1 System Overview

Without loss of generality, we present DORA using a disaggregated architecture, although it readily extends to the colocated architecture. As motivated in Section 2.2, our goal is to resolve the long-tail dilemma without paying either significant system overhead or additional algorithmic cost. DORA achieves this through three interlocking mechanisms:

- (i) *Multi-version streaming training* (Section 3.2) maintains multiple policy versions concurrently on rollout instances, enabling trajectory-level streaming that eliminates both intra-node and inter-node bubbles. Each trajectory is generated end-to-end under a single policy version, and the staleness across versions is bounded by a configurable window.
- (ii) *Dynamic resource orchestration* (Section 3.3) resolves the resource fragmentation inherent in multi-version rollout by dynamically re-partitioning DP groups and migrating requests, while preserving every sampled trajectory and respecting the staleness bound throughout.
- (iii) *KV-Cache reuse* (Section 3.4) turns the single-policy-per-trajectory design into a system-level advantage: because every token of a trajectory is generated under the same policy version, its KV-

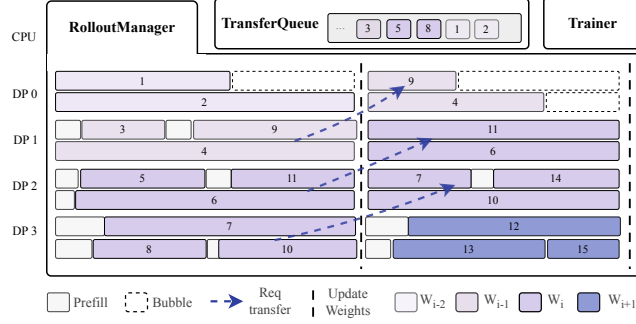


Figure 6: The execution timeline of DORA’s multi-version streaming training system.

Cache states are mathematically equivalent across any instance hosting that version, enabling nearly zero-re-prefill migration during request relocation.

As shown in Figure 6, these mechanisms cooperate at runtime through four cooperating components. A RolloutManager dispatches prompts to rollout instances, tagging each prompt with a policy version so that the entire trajectory is generated under that version. Completed trajectories stream into an asynchronous TransferQueue equipped with staleness monitoring. The Trainer consumes TBS samples for experience preparation and model training, then synchronizes the latest weights with rollout instances. A Load-balancing orchestrator monitors per-version workloads and triggers resource re-partitioning and request migration as needed, preserving the intermediate execution state. These components run on different nodes and coordinate via Remote Procedure Call (RPC), while workers running on accelerators execute the actual tasks.

Together, these mechanisms eliminate the long-tail bubble in rollout while preserving every sampled trajectory and bounding policy staleness—resolving the long-tail dilemma without significant system overhead or algorithmic compromise.

3.2 Multi-version Streaming Training

Trajectory-level streaming. DORA eliminates the synchronous barrier by streaming completed trajectories directly to training without waiting for straggling trajectories. During the rollout phase, we maintain multiple versions of policy weights in the rollout instances, where each Data Parallel (DP) group hosts a single version of the policy weights. At the onset of each step, we overprovide the rollout prompts, where $RBS > TBS$ generation requests are dispatched to the rollout instances. Training begins as soon as TBS samples are collected; unfinished long-tailed trajectories continue under their original policy version and flow into subsequent steps, ensuring the long trajectories are not abandoned or block the training. As illustrated in Figure 6, the Rollout and training processes execute non-blockingly; only after a training iteration concludes does the Trainer notify the RolloutManager to synchronize the latest weights.

Multi-version policy management. The key insight is that maintaining multiple policy versions concurrently allows long-tailed trajectories to continue under their original version while the training proceeds with completed trajectories. As illustrated in Figure 6, each prompt is tagged with a version w_j upon dispatch, ensuring $a_t \sim \pi_{w_j}(\cdot | s_t)$ for every token—so each trajectory is generated end-to-end under a single policy version without algorithmic modifications. For example, in Figure 6, Trajectory 4 (a long-tail request under version w_1) spans two training steps while Trajectories 1–3 complete and stream into training during Step 1. The system proceeds to Step 2 with updated weights w_2 without waiting for Trajectory 4 to complete, which continues under its original version w_1 in a dedicated DP group. This allows the legacy-version requests to execute in parallel, thereby eliminating the inter-node bubble and fully utilizing all rollout instances.

Sliding-window staleness control. Active versions are managed through a sliding window $W = \{w_j, \dots, w_{j-K+1}\}$ of size $|W| \leq K$. The window advancement follows a strict protocol to control the staleness. The window slides forward only when all trajectories from the oldest version w_{j-K+1} have been collected and forwarded to training. This provides a deterministic upper bound on policy

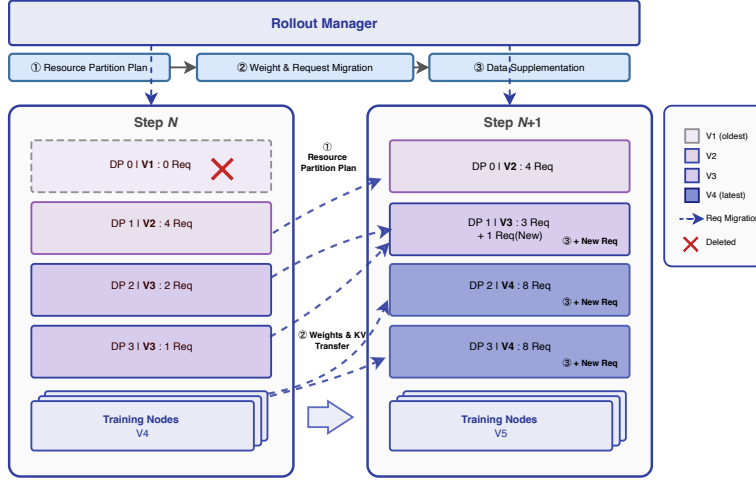


Figure 7: The workflow of the Dynamic Resource Orchestration.

staleness. The staleness bound K serves as an explicit control knob for the convergence–throughput tradeoff: a smaller K yields more on-policy data at the cost of rollout efficiency; a larger K increases throughput with controlled convergence impact.

Remaining challenges. While multi-version streaming preserves every sampled trajectory, bounds policy staleness, and partially alleviates both bubble types, it introduces two second-order efficiency challenges that motivate the subsequent mechanisms:

(i) *Resource fragmentation.* We observe that the legacy version’s pending requests decrease monotonically as trajectories complete, yet its allocated resources remain fixed, leading to notable device underutilization. This motivates the dynamic orchestration described in Section 3.3.

(ii) *Re-prefill overhead.* Migrating requests across DP groups naively re-triggers the full prefill phase—prohibitive for long-context scenarios (64k–128k tokens). This motivates the KV-Cache reuse mechanism in Section 3.4, which exploits the mathematical equivalence guaranteed by single-version trajectory generation.

3.3 Dynamic Resource Orchestration

Under multi-version streaming training, the pending request count of each legacy version w decreases monotonically as trajectories complete, yet the physical resources (DP groups) assigned to w remain fixed. This implies that static resource allocation leads to progressive underutilization—a legacy version’s DP groups may each serve only one or two residual requests, while the latest version, which carries the majority of new prompts, is under-provisioned. Therefore, DORA requires proactively rebalancing workloads while simultaneously controlling staleness.

To resolve this resource fragmentation, DORA employs a centralized orchestrator that dynamically re-partitions resources across model versions. The orchestrator maintains real-time metrics—active request counts per version, KV-Cache utilization, and generation progress—and supports three rebalancing triggers: (1) *update-driven*, mandatory upon the completion of each training step to promote the new policy version; (2) *utilization-based*, activated when KV-Cache pressure exceeds a threshold, avoiding costly eviction-induced recomputation; and (3) *temporal-based*, periodic execution to prevent orphan requests from lingering in legacy versions.

As illustrated in Figure 7, each re-balancing cycle executes three coordinated operations:

- **Resource partitioning plan.** The orchestrator assesses the distribution of active and pending requests across all maintained versions W to produce a migration plan. It computes the target DP group count for each version $w \in W$ proportional to its current workload, preventing resources from being stranded on legacy versions with dwindling tasks. This addresses the inter-instance data skewness identified in Section 2.2.

- **P2P weight update and request migration.** Once the migration plan is determined, the orchestrator generates a mapping from the current partition to the target partition. For each version whose allocation changes, it leverages P2P weight transfers to rescale the DP groups—decrease the legacy versions and increase the latest one. Active requests on re-assigned nodes are migrated to their new DP groups with execution states fully preserved via KV-Cache reuse (Section 3.4). Notably, no trajectory is abandoned during this process.
- **Staleness-aware data supplementation.** To control data staleness within the configured bound, the orchestrator prioritizes the latest policy version for proactive data injection, dispatching supplemental prompts until the *RBS* is fully met. This strategy maximizes sample freshness by ensuring that the majority of new trajectories are generated using the most recent model weights. Subsequently, to maintain high-watermark utilization across the cluster, the orchestrator performs opportunistic data injection following the request migration phase. Legacy versions are only supplemented with sufficient prompts to fill their residual idle slots. This tiered injection approach effectively saturates all rollout instances while preventing the over-production of stale trajectories, striking an optimal balance between hardware occupancy and algorithmic freshness.

This optimization cycle exemplifies algorithm-system co-design: **Proportional Resource Partitioning** resolves the resource fragmentation inherent in multi-version rollout; **Request Migration** preserves trajectory execution state without abandoning any sampled trajectory; and **Staleness-Aware Data Supplementation** ensures high hardware occupancy within the staleness bound.

3.4 KV-Cache Reuse

Note that request migration across DP groups naively re-triggers the prefill phase. The re-prefill cost scales with the growing context length and is further amplified in MoE architectures [17, 11], where it causes workload imbalance across non-MoE layers as shown in Section 2.2. However, DORA’s single-policy-per-trajectory design enables a powerful system-level optimization: since all tokens in a trajectory are generated by the same policy version π_w , the KV-Cache states are *mathematically equivalent* across any physical instance hosting version w . This equivalence enables cross-instance KV-Cache transfer that completely avoids re-prefill. Methods that mix multiple policy versions within a single trajectory forfeit this optimization: each weight update forces a full re-prefill of all ongoing trajectories, up to the output length. When the Load-Balancing Orchestrator triggers a resource re-allocation, DORA executes a coordinated two-phase state transfer:

- **Metadata forwarding.** Request metadata (request ID, generation state, decoded token count, and version tag) is transmitted via lightweight RPC. This control-plane transfer is negligible in both latency and bandwidth.
- **KV-Cache data transfer.** The voluminous KV Cache data—often comprising tens of gigabytes for long-context and MoE settings—is transferred using high-performance collective communication primitives, fully exploiting the available interconnect bandwidth.

Locality-aware scheduling. To minimize transfer volume, the orchestrator prioritizes re-assigning requests back to their original ranks when possible—preserving data locality and avoiding physical migration entirely. Only requests that must relocate due to version transitions incur transfer costs.

Hierarchical memory management. To alleviate VRAM pressure from aggregated requests during long-context training, DORA temporarily offloads KV-Caches to host memory [24], freeing device memory for active computations while preserving state for deferred generation. This hierarchical management safeguards system efficiency even under extreme long-tailed workloads.

By eliminating re-prefill from request migration, KV-Cache reuse closes the last source of system overhead introduced by multi-version coexistence: long-tailed trajectories now traverse legacy DP groups, get migrated under load-balancing, and continue generation—all without the prefill recomputation that would otherwise grow dramatically with context length.

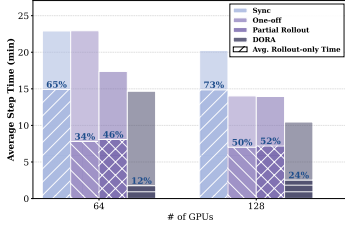


Figure 8: Average RL step time across different training paradigms on Dense-32B.

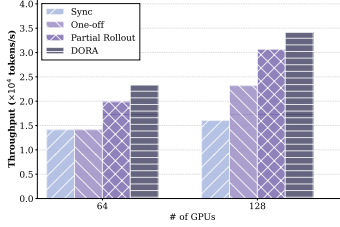


Figure 9: End-to-End Throughput across different training paradigms on Dense-32B.

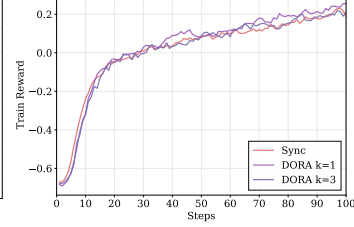


Figure 10: Training reward scores comparison for various training paradigms.

4 Experiments

4.1 Experimental Setup

We evaluate DORA on a 16-node H800 cluster (128 GPUs) for open-source experiments and on a production cluster of non-CUDA accelerators for large-scale evaluation. Two model scales are used: Qwen2.5-32B [26] for dense architectures and a ~ 500 B-parameter MoE model for production scale. We compare DORA against three representative paradigms spanning the constraint–efficiency landscape: (1) *Synchronous* (all constraints satisfied, batch-barrier limited); (2) *One-step off-policy* (overlaps stages but does not eliminate rollout bubbles); and (3) *Partial rollout in the colocated model placement* (eliminates bubbles by relaxing single-version generation, requiring algorithmic corrections). All baselines are implemented in the same in-house framework on identical hardware. Full hardware specs, software stack, dataset, and training hyperparameters are in Appendix B.

4.2 Training Performance

Rollout Acceleration. DORA achieves consistent gains across both 64 and 128 GPUs. As shown in Figure 8, on 64 GPUs, DORA shrinks the rollout-only phase—the portion that cannot be overlapped with training—from 65% of the step time under synchronous training to merely 12%, an $8.2\times$ reduction in absolute duration (14.9 min $\rightarrow$ 1.8 min). The end-to-end step time correspondingly drops by $1.56\times$. This compression directly resolves the long-tail dilemma identified in Section 1: long-tailed trajectories continue under their legacy policy versions in dedicated DP groups while new requests saturate the released resources, eliminating both the intra- and inter-node bubbles. Compared with partial rollout, the strongest long-tail-mitigating baseline, DORA still achieves $1.18\times$ end-to-end speedup, owing to its zero-re-prefill migration (Section 3.4) that avoids the re-prefill cost partial rollout incurs at each weight update. The same pattern holds on 128 GPUs—Sync’s rollout-only fraction grows to 73% while DORA’s remains at only 24%—yielding $5.9\times$ rollout and $1.93\times$ end-to-end speedup¹.

End-to-End Throughput. The step-time gains translate into proportional throughput improvements across both cluster scales. As shown in Figure 9, on 64 GPUs, DORA reaches **23,327 tokens/s**, a $1.65\times$ improvement over synchronous training and $1.17\times$ over partial rollout. The same pattern holds on 128 GPUs, where DORA achieves **34,135 tokens/s**, yielding $2.12\times$ and $1.11\times$ speedups over the two baselines, respectively. Notably, this throughput reflects the trajectories actually consumed by training. Both baselines waste accelerator time—one-step off-policy through long-tail idleness, partial rollout through re-prefill at each weight update—whereas DORA’s multi-version streaming and zero-re-prefill migration convert all rollout duration into effective throughput.

Model Convergence. To verify that DORA’s efficiency gains do not compromise algorithmic fidelity, we monitor the mean training reward over 100 steps on 72 GPUs (Figure 10). Both DORA variants ($k=1$ and $k=3$) closely track the synchronous baseline, confirming that multi-version streaming training preserves convergence behavior in the bounded staleness settings. Nevertheless, we observe that the $k=3$ variant exhibits a moderately slower convergence rate compared to $k=1$, which further underscores the necessity of staleness control.

¹Theoretically, the rollout-only phase can be fully eliminated via aggressive staleness strategies as detailed in Appendix C.3.

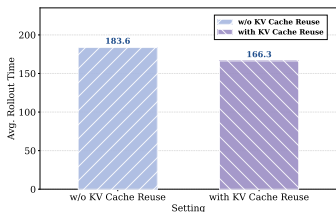


Figure 11: Ablation study on the effect of KV Cache reuse on average rollout time (in seconds).

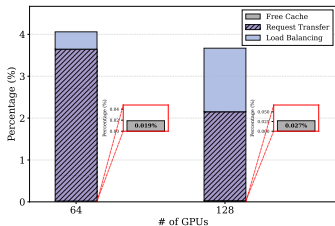


Figure 12: System overhead breakdown for DORA on 64 and 128 GPUs.

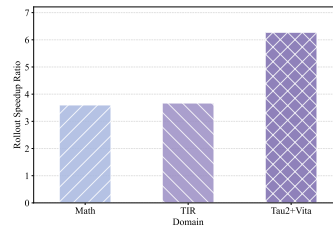


Figure 13: DORA vs. sync in production on a $\sim 500\text{B}$ MoE model with 64K max resp. len.

4.3 Ablation Study

We conduct an ablation study to further investigate the contribution of each component to DORA’s overall acceleration. As shown in Figure 11, the baseline without KV cache reuse requires an average of 183 seconds per rollout, whereas the variant with KV cache reuse completes the same rollout in approximately 166 seconds under identical dense 32B model settings. This KV cache optimization yields a 9% speedup over the non-cached baseline. We do not ablate the dynamic resource orchestration module, as it is indispensable for the system to function.

4.4 Overhead Analysis

We quantify the three primary overheads introduced by DORA’s dynamic orchestration and KV-Cache reuse: P2P-based load balancing, request transfer, and free-cache operations. As shown in Figure 12, all three remain well below the throughput gains they enable. Load balancing—covering request monitoring, resource re-partitioning, and P2P weight synchronization—accounts for 0.4% / 1.5% of total execution time at 64 / 128 GPUs. Request transfer, which carries metadata and physical KV-Cache states, is bounded under 4% and *decreases* at scale (3.6% $\rightarrow$ 2.1%) as larger throughput amortizes the migration cost. Free-cache operations are negligible (under 0.03% in both settings), confirming that our hierarchical memory management runs entirely off the critical path. Overall, the aggregate system overhead does not grow with cluster size.

4.5 Production Deployment

We further deploy DORA on a $\sim 500\text{B}$ -parameter MoE model with up to 64K-token responses, comparing against the well-tuned synchronous baseline used in production (since running all baselines at this scale is prohibitive) with 4,096 accelerators. As shown in Figure 13, DORA achieves $3.6\times$ rollout speedup on mathematical and tool-integrated reasoning, and up to $6.2\times$ on agentic training over Tau2-bench [4] and Vita [15]. The widening gap on agentic workloads—where responses are longest and most skewed—aligns with DORA’s design hypothesis: the more pronounced the long tail, the larger the bubble that multi-version streaming eliminates. DORA has served as the default asynchronous paradigm in our in-house RL framework since 2025, delivering $2 - 4\times$ end-to-end speedup with no quality degradation at scale, powering our competitive open-source LLMs.

5 Conclusion and Limitations

We present DORA, a scalable asynchronous RL system that resolves the long-tail dilemma in large-scale LLM post-training. By embodying a *multi-version rollout* paradigm, where multiple policy versions coexist and each trajectory is generated end-to-end under a single version, DORA eliminates the long-tail bubble without introducing algorithmic corrections. Furthermore, this single-policy generation yields KV-Cache equivalence, enabling zero-re-prefill migration during request relocation. Experiments demonstrate up to $8.2\times$ rollout speedup and $2.12\times$ end-to-end acceleration over synchronous training, validated by large-scale industrial deployments. Despite these significant efficiency gains, DORA has limitations that merit further investigation. Algorithmically, the staleness bound K requires manual configuration, relying on PPO’s clipping to mitigate off-policy bias; incorporating adaptive staleness control or explicit delay compensation could further optimize the

convergence–throughput tradeoff. Experimentally, our evaluations were conducted within a controlled in-house framework, and MoE scaling was validated primarily on production data and cluster. Future efforts will address this by benchmarking directly against public systems such as veRL [33] and AReal [8], alongside extensive evaluations on more open-source MoE models.

References

- [1] Joshua Achiam, David Held, Aviv Tamar, and Pieter Abbeel. Constrained policy optimization. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pages 22–31. PMLR, 2017.
- [2] Arash Ahmadian, Chris Cremer, Matthias Gallé, Marzieh Fadaee, Julia Kreutzer, Olivier Pietquin, Ahmet Üstün, and Sara Hooker. Back to basics: Revisiting REINFORCE-style optimization for learning from human feedback in LLMs. *arXiv preprint arXiv:2402.14740*, 2024.
- [3] Anthropic. Introducing claude opus 4.5, 2025. URL <https://www.anthropic.com/news/claude-opus-4-5>.
- [4] Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik Narasimhan. τ^2 -bench: Evaluating conversational agents in a dual-control environment. *CoRR*, abs/2506.07982, 2025.
- [5] Pritam Damania, Shen Li, Alban Desmaison, Alisson Azzolini, Brian Vaughan, Edward Yang, Gregory Chanan, Guoqiang Jerry Chen, Hongyi Jia, Howard Huang, et al. Pytorch rpc: Distributed deep learning built on tensor-optimized remote procedure calls. *Proceedings of Machine Learning and Systems*, 5: 219–231, 2023.
- [6] Dong Du, Shulin Liu, Tao Yang, Shaohua Chen, and Yang Li. Ulorl: An ultra-long output reinforcement learning approach for advancing large language models’ reasoning abilities. *arXiv preprint arXiv:2507.19766*, 2025.
- [7] Lasse Espeholt, Hubert Soyer, Remi Munos, Karen Simonyan, Volodymyr Mnih, Tom Ward, Yotam Doron, Vlad Firoiu, Tim Harley, Iain Dunning, Shane Legg, and Koray Kavukcuoglu. IMPALA: Scalable distributed deep-RL with importance weighted actor-learner architectures. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, pages 1407–1416. PMLR, 2018.
- [8] Wei Fu, Jiaxuan Gao, Xujie Shen, Chen Zhu, Zhiyu Mei, Chuyi He, Shusheng Xu, Guo Wei, Jun Mei, Jiashu Wang, et al. Areal: A large-scale asynchronous reinforcement learning system for language reasoning. *arXiv preprint arXiv:2505.24298*, 2025.
- [9] Wei Gao, Yuheng Zhao, Dakai An, Tianyuan Wu, Lunxi Cao, Shaopan Xiong, Ju Huang, Weixun Wang, Siran Yang, Wenbo Su, et al. Rollpacker: Mitigating long-tail rollouts for fast, synchronous rl post-training. *arXiv preprint arXiv:2509.21009*, 2025.
- [10] Evan Greensmith, Peter L. Bartlett, and Jonathan Baxter. Variance reduction techniques for gradient estimates in reinforcement learning. *Journal of Machine Learning Research*, 5:1471–1530, 2004.
- [11] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shironi Ma, Xiao Bi, et al. Deepseek-r1 incentivizes reasoning in llms through reinforcement learning. *Nature*, 645(8081):633–638, 2025.
- [12] Zhenyu Han, Ansheng You, Haibo Wang, Kui Luo, Guang Yang, Wenqi Shi, Menglong Chen, Sicheng Zhang, Zeshun Lan, Chunshi Deng, et al. Asyncflow: An asynchronous streaming rl framework for efficient llm post-training. *arXiv preprint arXiv:2507.01663*, 2025.
- [13] Yaru Hao, Li Dong, Xun Wu, Shaohan Huang, Zewen Chi, and Furu Wei. On-policy rl with optimal reward baseline, 2025. URL <https://arxiv.org/abs/2505.23585>.
- [14] Jingkai He, Tianjian Li, Erhu Feng, Dong Du, Qian Liu, Tao Liu, Yubin Xia, and Haibo Chen. History rhymes: Accelerating llm reinforcement learning with rhymerrl. *arXiv preprint arXiv:2508.18588*, 2025.
- [15] Wei He, Yueqing Sun, Hongyan Hao, Xueyuan Hao, Zhikang Xia, Qi Gu, Chengcheng Han, Dengchang Zhao, Hui Su, Kefeng Zhang, Man Gao, Xi Su, Xiaodong Cai, Xunliang Cai, Yu Yang, and Yunke Zhao. Vitabench: Benchmarking LLM agents with versatile interactive tasks in real-world applications. *CoRR*, abs/2509.26490, 2025.
- [16] Jian Hu, Xibin Wu, Zilin Zhu, Xianyu, Weixun Wang, Dehao Zhang, and Yu Cao. Openrlhf: An easy-to-use, scalable and high-performance rlhf framework. *arXiv preprint arXiv:2405.11143*, 2024.

- [17] Albert Q Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al. Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024.
- [18] Sham Kakade and John Langford. Approximately optimal approximate reinforcement learning. In *Proceedings of the 19th International Conference on Machine Learning (ICML)*, pages 267–274, 2002.
- [19] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, 2023.
- [20] Xiangru Lian, Yijun Huang, Yuncheng Li, and Ji Liu. Asynchronous parallel stochastic gradient for nonconvex optimization. *Advances in neural information processing systems*, 28, 2015.
- [21] Michael Luo, Sijun Tan, Roy Huang, Ameen Patel, Alpay Ariyak, Qingyang Wu, Xiaoxiang Shi, Rachel Xin, Colin Cai, Maurice Weber, et al. Deepcoder: A fully open-source 14b coder at o3-mini level. *Notion Blog*, 1, 2025.
- [22] Michael Noukhovitch, Shengyi Huang, Sophie Xhonneux, Arian Hosseini, Rishabh Agarwal, and Aaron Courville. Asynchronous rlhf: Faster and more efficient off-policy rl for language models. *arXiv preprint arXiv:2410.18252*, 2024.
- [23] OpenAI. Introducing openai o1, 2024. URL <https://openai.com/o1/>.
- [24] Ruoyu Qin, Zheming Li, Weiran He, Jialei Cui, Heyi Tang, Feng Ren, Teng Ma, Shangming Cai, Yineng Zhang, Mingxing Zhang, et al. Mooncake: A kvcache-centric disaggregated architecture for llm serving. *ACM Transactions on Storage*, 2024.
- [25] James Queeney, Ioannis Ch. Paschalidis, and Christos G. Cassandras. Generalized proximal policy optimization with sample reuse. *Advances in Neural Information Processing Systems (NeurIPS)*, 34, 2021.
- [26] Qwen, :, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tianyi Tang, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. Qwen2.5 technical report, 2025. URL <https://arxiv.org/abs/2412.15115>.
- [27] John Schulman, Sergey Levine, Pieter Abbeel, Michael Jordan, and Philipp Moritz. Trust region policy optimization. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, pages 1889–1897. PMLR, 2015.
- [28] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [29] ByteDance Seed, Jiaze Chen, Tiantian Fan, Xin Liu, Lingjun Liu, Zhiqi Lin, Mingxuan Wang, Chengyi Wang, Xiangpeng Wei, Wenyuan Xu, et al. Seed1. 5-thinking: Advancing superb reasoning models with reinforcement learning. *arXiv preprint arXiv:2504.13914*, 2025.
- [30] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Mingchuan Zhang, YK Li, Y Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- [31] Zhihong Shao, Yuxiang Luo, Chengda Lu, Z. Z. Ren, Jiewen Hu, Tian Ye, Zhibin Gou, Shirong Ma, and Xiaokang Zhang. Deepseekmath-v2: Towards self-verifiable mathematical reasoning. *CoRR*, abs/2511.22570, 2025.
- [32] Guangming Sheng, Yuxuan Tong, Borui Wan, Wang Zhang, Chaobo Jia, Xibin Wu, Yuqi Wu, Xiang Li, Chi Zhang, Yanghua Peng, et al. Laminar: A scalable asynchronous rl post-training framework. *arXiv preprint arXiv:2510.12633*, 2025.
- [33] Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. Hybridflow: A flexible and efficient rlhf framework. In *Proceedings of the Twentieth European Conference on Computer Systems*, pages 1279–1297, 2025.
- [34] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.

- [35] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling llm test-time compute optimally can be more effective than scaling model parameters. *arXiv preprint arXiv:2408.03314*, 2024.
- [36] Kimi Team, Angang Du, Bofei Gao, Bowei Xing, Changjiu Jiang, Cheng Chen, Cheng Li, Chenjun Xiao, Chenzhuang Du, Chonghua Liao, et al. Kimi k1. 5: Scaling reinforcement learning with llms. *arXiv preprint arXiv:2501.12599*, 2025.
- [37] Meituan LongCat Team, Anchun Gui, Bei Li, Bingyang Tao, Bole Zhou, Borun Chen, Chao Zhang, Chengcheng Han, Chenhui Yang, Chi Zhang, et al. Introducing longcat-flash-thinking: A technical report. *arXiv preprint arXiv:2509.18883*, 2025.
- [38] Meituan LongCat Team, Bei Li, Bingye Lei, Bo Wang, Bolin Rong, Chao Wang, Chao Zhang, Chen Gao, Chen Zhang, Cheng Sun, et al. Longcat-flash technical report. *arXiv preprint arXiv:2509.01322*, 2025.
- [39] Meituan LongCat Team, Anchun Gui, Bei Li, Bingyang Tao, Bole Zhou, Borun Chen, Chao Zhang, Chen Gao, Chen Zhang, Chengcheng Han, et al. Longcat-flash-thinking-2601 technical report. *arXiv preprint arXiv:2601.16725*, 2026.
- [40] Lex Weaver and Nigel Tao. The optimal reward baseline for gradient-based reinforcement learning. In *Proceedings of the 17th Conference in Uncertainty in Artificial Intelligence*, pages 538–545, 2001.
- [41] Bo Wu, Sid Wang, Yunhao Tang, Jia Ding, Eryk Helenowski, Liang Tan, Tengyu Xu, Tushar Gowda, Zhengxing Chen, Chen Zhu, et al. Llamarl: A distributed asynchronous reinforcement learning framework for efficient large-scale llm training. *arXiv preprint arXiv:2505.24034*, 2025.
- [42] Youshao Xiao, Zhenglei Zhou, Fagui Mao, Weichang Wu, Shangchun Zhao, Lin Ju, Lei Liang, Xiaolu Zhang, and Jun Zhou. An adaptive placement and parallelism framework for accelerating rlhf training. *arXiv preprint arXiv:2312.11819*, 2023.
- [43] Zhewei Yao, Reza Yazdani Aminabadi, Olatunji Ruwase, Samyam Rajbhandari, Xiaoxia Wu, Ammar Ahmad Awan, Jeff Rasley, Minjia Zhang, Conglong Li, Connor Holmes, et al. Deepspeed-chat: Easy, fast and affordable rlhf training of chatgpt-like models at all scales. *arXiv preprint arXiv:2308.01320*, 2023.
- [44] Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, et al. Dapo: An open-source llm reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.
- [45] Yiqi Zhang, Huiqiang Jiang, Xufang Luo, Zhihe Yang, Chengruidong Zhang, Yifei Shen, Dongsheng Li, Yuqing Yang, Lili Qiu, and Yang You. Sortedrl: Accelerating rl training for llms through online length-aware scheduling. In *ES-FoMo III: 3rd Workshop on Efficient Systems for Foundation Models*, 2025.
- [46] Shuxin Zheng, Qi Meng, Taifeng Wang, Wei Chen, Nenghai Yu, Zhi-Ming Ma, and Tie-Yan Liu. Asynchronous stochastic gradient descent with delay compensation. In *International conference on machine learning*, pages 4120–4129. PMLR, 2017.
- [47] Yinmin Zhong, Zili Zhang, Xiaoni Song, Hanpeng Hu, Chao Jin, Bingyang Wu, Nuo Chen, Yukun Chen, Yu Zhou, Changyi Wan, et al. Streamrl: Scalable, heterogeneous, and elastic rl for llms with disaggregated stream generation. *arXiv preprint arXiv:2504.15930*, 2025.
- [48] Zilin Zhu, Chengxing Xie, Xin Lv, and slime Contributors. slime: An llm post-training framework for rl scaling. <https://github.com/THUDM/slime>, 2025. GitHub repository. Corresponding author: Xin Lv.

A Related Work

To accelerate reinforcement learning (RL) post-training for large language models, various distributed training systems and asynchronous strategies have been proposed. Previous RL training systems [43, 42, 16, 33] primarily focus on model placement and scheduling within a synchronous training paradigm. Synchronous training cycles sequentially through rollout, experience preparation, and model training within each step. While this provides clean algorithmic semantics by enforcing a strict batch barrier between rollout and training, it suffers from severe batch-barrier idle time—especially in long-context and complex reasoning scenarios where the generation length is highly skewed. To alleviate this rollout bottleneck, recent efforts have shifted towards asynchronous and semi-asynchronous paradigms, which can be broadly categorized into three main directions.

Replication-based (Oversampling) Methods. Replication-based or oversampling methods [9, 45] attempt to shorten the rollout duration by over-provisioning generation prompts. Specifically, they dispatch a larger number of rollout requests than the required training batch size and simply discard the in-flight long trajectories once enough completed responses are collected. While this approach effectively reduces the long-tail latency, it fundamentally compromises data integrity. The discarded trajectories often contain the lengthy chain-of-thought reasoning steps that are crucial for developing emergent agentic capabilities. Furthermore, the resulting length-biased distribution distorts advantage estimation, particularly in group-relative algorithms like GRPO [30], whereas DORA preserves every sampled trajectory without algorithmic distortion.

One-step Off-policy Methods. K -step off-policy methods [21, 47, 22, 14, 12]—most commonly one-step off-policy—overlap the rollout and training stages temporally. They achieve this by allowing the current step’s rollout to use behavior policy weights from the previous iteration while the training stage updates the target policy. Although this design successfully hides the pipeline bubble between the rollout and training stages, it does not address the fundamental duration of the rollout phase itself. Both intra-node bubbles (idle slots within a device) and inter-node bubbles (faster instances waiting for the slowest) persist under highly skewed workload distributions. In contrast, DORA utilizes a multi-version streaming mechanism that completely breaks the synchronous batch barrier, directly resolving these intra- and inter-node inefficiencies.

Partial-rollout Methods. Partial-rollout methods [36, 41, 6, 8, 48] mitigate the long-tail issue by segmenting lengthy responses into smaller chunks at each weight update. When a new policy version becomes available, ongoing trajectories are resumed and continued under the latest weights. This approach effectively eliminates device idleness but introduces two significant challenges. First, from an algorithmic perspective, a single trajectory is now stitched together from multiple distinct policy versions. This departs from the standard RL formulation and necessitates complex algorithmic corrections, such as masking earlier segments during loss computation [36] or applying decoupled PPO objectives [8] to maintain convergence. Second, from a system perspective, every weight update invalidates the KV-Cache for the ongoing trajectories, forcing a full re-prefill of the context. This re-prefill overhead grows dramatically with context length and is particularly prohibitive in MoE architectures. DORA bypasses both challenges by ensuring each trajectory is generated end-to-end under a single policy version, preserving standard algorithmic semantics and enabling zero-re-prefill migration.

Concurrently with our work, several systems [29, 32] have explored multi-version streaming training concepts. However, these systems rely on two-tier CPU relay architectures for weight management and employ significantly different mechanisms for workload orchestration and staleness control compared to DORA’s centralized load-balancing orchestrator and zero-re-prefill migration.

B Experimental Setup

Testbed. Our experiments are conducted on a cluster consisting of 16 nodes, each equipped with 8 NVIDIA H800 GPUs. Intra-node communication is facilitated by NVLink with a bandwidth of 400 GB/s, while inter-node connectivity is provided by 8×400 Gbps network interfaces. Additionally, our production cluster employs non-CUDA accelerators, each providing approximately 60 GB of available device memory.

Models and Metrics. Our experiments use Qwen2.5-32B [26] for dense architectures and a around 500B open-source MoE model for MoE architectures. We measure *end-to-end throughput* (tokens/s), calculated as the total tokens (prompts and responses) processed per second, and *average step time* (min), which represents the wall-clock time per RL iteration. All reported numbers are averaged over five RL iterations after the warm-up phase to reflect steady-state performance.

Datasets. We utilize the "DAPO-Math-17k" dataset for training, with the maximum input and output sequence lengths set to 2K and 30K tokens, respectively.

Baselines and Implementation. We compare DORA against three representative RL training paradigms: (1) *Synchronous (All-Colocated)*, which satisfies all constraints but suffers from batch barriers; (2) *One-step off-policy*, which satisfies all constraints with staleness $K=1$ but only overlaps pipeline stages without eliminating rollout bubbles; and (3) *Partial rollout* in All-colocated implementation similar to the work [36]. All baselines are implemented within the same in-house RL framework to ensure a controlled comparison under identical hardware and software configurations. Our RL system uses vLLM [19] as the inference engine, Megatron-LM [34] as the training backend, and extends torch RPC [5] with streaming primitives. The software environment includes CUDA-12.4, PyTorch-2.6.0, vLLM-0.8.5, and NCCL-2.28.

Training Configurations. For the RL algorithm, we follow the setting of DAPO [44], a variant of GRPO. Each rollout consists of a prompt batch size of 512, with 16 responses sampled per prompt, resulting in a global training batch size of 8,192. Each training iteration involves 16 update steps with a micro-batch size of 512. To stress-test efficiency under realistic long-tailed generation, we select an intermediate checkpoint where the mean response length is 2.4K tokens and the maximum reaches 30K.

C Case Study: Empirical Validation of Multi-Version Streaming

This part presents a detailed empirical case study that validates the algorithmic properties of DORA’s multi-version streaming training. Using rollout logs spanning 22,820 records (365,120 responses) over training steps 20–100 with staleness bound $K=3$, we examine three modes: *DORA*, *Partial Rollout*, and *Synchronous* training. Each record contains 16 responses scored by a binary reward model (+1/−1).

C.1 Observation 1: Bounded Staleness Incurs Nearly Zero Quality Degradation

A natural concern with asynchronous training is that stale data degrades learning. We perform a controlled analysis to disentangle staleness from confounding variables.

Naïve observation. Aggregating across all records, staleness-0 data achieves a 58.9% pass rate versus 55.6% for staleness-1, an apparent 3.3 percentage point gap.

Controlled analysis. This gap is entirely explained by *selection bias*: harder problems produce longer responses, which take more time to generate and thus receive higher staleness labels. Table 1 stratifies records by difficulty (pass rate bin) and compares staleness-0 versus staleness-1 *within each stratum*:

Table 1: Pass rate (%) by staleness, stratified by problem difficulty (DORA, $K=3$). The staleness gap vanishes after controlling for difficulty, confirming zero quality degradation.

Difficulty Bin	s=0 (%)	n (s=0)	s=1 (%)	n (s=1)	Δ (%)
Hard (PR < 0.25)	10.93	187	11.27	667	+0.34
Medium (0.25–0.5)	34.42	146	34.39	580	−0.03
Medium-Easy (0.5–0.75)	60.67	184	60.05	625	−0.62
Easy (PR > 0.75)	86.55	441	86.55	1263	+0.00

We further verify this at the per-response level using Partial Rollout’s generation segments (Table 2), which provides 141,632 responses with explicit version labels:

Mechanism. Figure 14(b) confirms the causal pathway: staleness-1 responses have a median token count of 1,495 versus 1,015 for staleness-0 (47% longer). Longer generation implies harder problems;

Table 2: Per-response staleness analysis (Partial Rollout, $n=141,632$). Within each difficulty bin, staleness-0 and staleness-1 responses perform identically.

Pass Rate Bin	s=0 (%)	s=1 (%)	Δ (%)	n (total)
[0, 0.125)	6.25	6.25	0.00	14,384
[0.125, 0.25)	15.43	15.28	-0.15	17,776
[0.25, 0.5)	34.04	33.86	-0.17	25,712
[0.5, 0.75)	60.18	59.85	-0.33	28,880
[0.75, 0.875)	78.37	78.63	+0.26	19,344
[0.875, 1.0]	91.37	91.28	-0.10	35,520

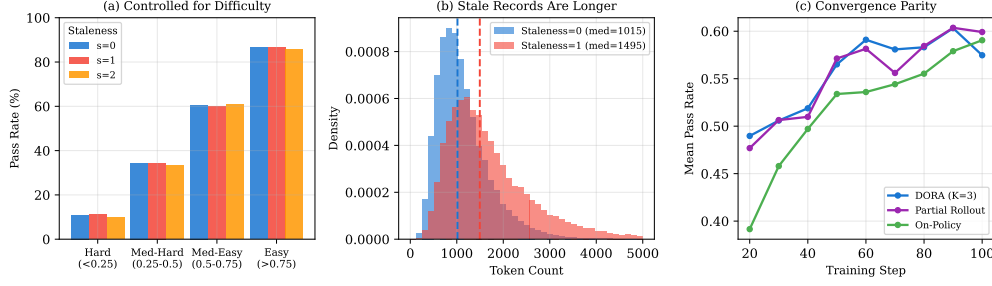


Figure 14: (a) After controlling for problem difficulty, the staleness gap vanishes ($\Delta < 0.6\%$ across all bins). (b) Selection bias mechanism: stale records are systematically longer (harder). (c) All three paradigms converge to the same pass-rate range (0.57–0.60).

under DORA’s asynchronous scheduling, these naturally complete after a weight update and receive a higher staleness label. The *on-policy* improvement rate is only 0.113%/step (measured from on-policy data over steps 50–100), so the theoretical maximum penalty for $K=3$ is merely $3 \times 0.113 = 0.34\%$, well below sampling noise. This validates that DORA’s sliding-window staleness control (Section 3.2) is sufficient for convergence parity.

C.2 Observation 2: Long-Tail Trajectories Carry the Highest Training Signal

DORA’s trajectory-level streaming preserves every sampled trajectory (Section 3.2), in contrast to replication-based methods that discard long-tail samples once enough short ones complete. We validate the importance of this design by measuring the GRPO gradient signal, quantified as within-group reward variance, across difficulty levels:

Table 3: Reward variance (gradient signal strength for GRPO) by difficulty. Medium-difficulty problems with the highest variance are also the longest and most likely to be stale, i.e., exactly the trajectories that replication-based methods discard first.

Difficulty	n	Reward Var.	Avg Length	Avg Staleness
Hard (1–3/16 correct)	1,007	0.409	3,755	0.99
Medium-Hard (4–7/16)	868	0.939	3,624	1.01
Medium-Easy (8–11/16)	950	1.001	3,418	0.96
Easy (12–14/16)	1,206	0.617	3,025	0.90
Very Easy (15–16/16)	721	0.250	2,711	0.87

The medium-difficulty records (pass rate 0.25–0.75) exhibit the highest reward variance (~ 1.0), providing the strongest gradient signal. These records are simultaneously the longest (3,400–3,800 chars) and the most stale (staleness 0.96–1.01). In a replication-based system that terminates generation once *TBS* short trajectories are collected, these high-signal samples would be the first to be discarded. DORA preserves all of them via multi-version streaming, ensuring the integrity of advantage estimation in GRPO.

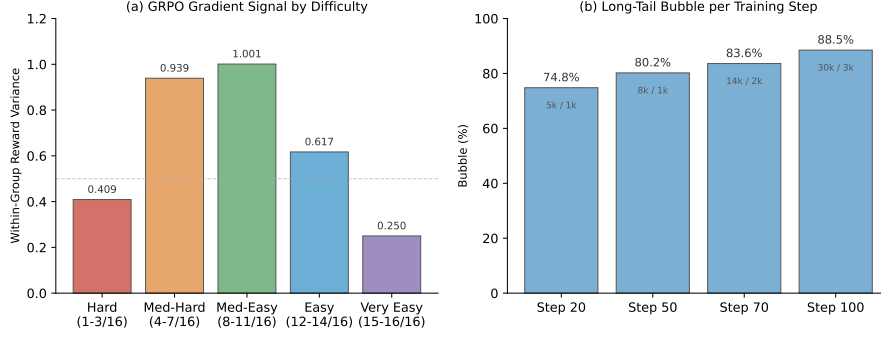


Figure 15: (a) GRPO gradient signal peaks at medium difficulty (pass rate 0.25–0.75). (b) Under synchronous training, the inter-node bubble exceeds 75% across all steps and grows to 88.5% at step 100 as responses lengthen.

C.3 Observation 3: Quantifying the Long-Tail Bubble

We directly measure the inter-node bubble that DORA could theoretically eliminate from the view of the request workloads. For each training step, we compute the fraction of decode time wasted waiting for the longest trajectory:

$$\text{Bubble} = \frac{\max_i L_i - \overline{L_i}}{\max_i L_i}$$

As shown in Figure 15(b), the bubble exceeds 74% at step 20 and grows to **88.5%** at step 100 (max response: 30,720 tokens vs. mean: 3,536). The max-to-median token ratio reaches **28×**, consistent with the order-of-magnitude skew reported in Section 2.2. This quantifies the efficiency opportunity that DORA captures through multi-version streaming: by allowing the longest trajectories to continue under their legacy version while new requests run on the latest policy model, DORA could theoretically eliminate this 75–89% bubble entirely from the viewpoint of engineering only.

C.4 Summary

This case study empirically characterizes DORA’s multi-version streaming training along three axes. Two concern its algorithmic design principles, and one quantifies the efficiency opportunity that motivates the design.

- **Bounded staleness (design principle).** After controlling for problem difficulty, staleness ≤ 3 produces nearly zero measurable quality degradation ($\Delta < 0.6\%$ across all strata); the apparent staleness gap is entirely explained by selection bias.
- **Preserving every sampled trajectory (design principle).** The longest, most stale trajectories carry the highest GRPO gradient signal (reward variance ~ 1.0). Discarding them, as replication-based methods do, removes the most valuable training data.
- **Quantifying the long-tail bubble (motivation).** Under synchronous training the inter-node bubble reaches 75–89%, confirming that the inefficiency DORA targets is substantial.

Together, these observations support DORA’s design goal of resolving the long-tail dilemma without introducing algorithmic corrections.

D A Policy-Improvement View of Single-Version Rollout in DORA vs. Partial Rollout

This appendix examines the algorithmic dimension of DORA’s single-policy-per-trajectory design through its effect on the *policy-improvement guarantee* underlying PPO-style updates; a complementary variance-based analysis follows in Appendix F. In asynchronous training a batch mixes trajectories from multiple policy versions, and DORA and partial-rollout methods [36, 41, 6, 8] differ in how such a batch enters this guarantee. We examine this through the classical trust-region

/ sample-reuse framework for monotonic policy improvement [27, 1, 25]. Our goal is modest: to locate DORA and partial rollout as two regimes of a common mixed-behavior formulation, and to identify the structural consequence—absence versus presence of an advantage-substitution bias—that distinguishes them.

Scope and abstraction. The analysis below is a *stylized comparison under a common formulation*, not a faithful model of either system’s full training dynamics. We adopt the *mixed-behavior-policy formulation*, in which the policy version index is lifted into the state [7]: a batch drawn from lagged versions $\{\pi_{w_k}\}$ is described by an augmented state $(s, i) \in \mathcal{S} \times \mathcal{I}$, a mixed behavior policy $\beta(a | s, i) := \pi_{w_i}(a | s)$, and an *index-transition kernel* $q(i' | i)$ that governs how the active version evolves along a trajectory. Within this formulation, the two asynchronous paradigms are the two representative values of q :

- **Trajectory-level mixing (DORA).** Each trajectory is generated end-to-end under the version active at its dispatch, so the index never changes: $q(i' | i) = \mathbf{1}\{i' = i\}$.
- **Step/segment-level mixing (partial rollout).** A trajectory may switch to a newer version at a weight update, with switching probability $\sigma > 0$: $q(i' | i) = (1 - \sigma) \mathbf{1}\{i' = i\} + \sigma \kappa(i' | i)$, where κ is the target-version distribution.

This formulation is faithful to DORA’s actual generation process, which tags each prompt with a single version upon dispatch (Section 3). It uses $\sigma > 0$ as a stylized representative of the step/segment-level family; it does not model the additional correction mechanisms that concrete partial-rollout systems apply to their stitched trajectories—decoupled objectives, importance corrections across segments, gradient masking, and so on—which lie outside our scope. Throughout, π_θ denotes the target policy, $\{\pi_{w_k}\}_{k=1}^K$ the lagged policies with schedule $\{p_k\}$, and $\mu_k = J(\pi_{w_k})$ their expected rewards.

D.1 Setup and Notation

The phenomenon studied here—mid-trajectory version switching—is intrinsically token-level: a weight update can change the active policy version between two tokens of the same response. We therefore adopt a token-level MDP in which a response is a length- H trajectory $(a_1, \dots, a_H)$ and the active version may evolve along it. Let $J(\pi) = \mathbb{E}[\sum_{t \geq 0} \gamma^t r(s_t, a_t) | \pi]$, let d_π denote the discounted state-visitation distribution, and let A^π denote the advantage function. For a behavior policy π_w , define the importance-weighted surrogate objective

$$L_{\pi_w}(\pi_\theta) := \frac{1}{1 - \gamma} \mathbb{E}_{s \sim d_{\pi_w}, a \sim \pi_w(\cdot | s)} \left[\frac{\pi_\theta(a | s)}{\pi_w(a | s)} A^{\pi_w}(s, a) \right], \quad (4)$$

and the expected-advantage coefficient $C_{\pi_\theta, \pi_w} := \max_s |\mathbb{E}_{a \sim \pi_\theta} [A^{\pi_w}(s, a)]|$. On the augmented state space, the mixed behavior policy β induces a visitation distribution $d_\beta(s, i)$ and an advantage $A^\beta((s, i), a)$; the index-transition kernel q enters only through the future evolution of i . We write $J_{\text{mix}} := J(\beta)$ for the return of the mixed behavior policy, and $C_{\pi_\theta, \beta} := \max_{(s, i)} |\mathbb{E}_{a \sim \pi_\theta} [A^\beta((s, i), a)]|$.

D.2 Monotonic Improvement under Trajectory-Level Mixing

Under trajectory-level mixing the augmented-state advantage collapses to the per-version advantage, which lets the standard sample-reuse bound apply verbatim.

Lemma 1 (Advantage reduction under trajectory-level mixing) *If $q(i' | i) = \mathbf{1}\{i' = i\}$, then $d_\beta(s, i) = \alpha_i d_{\pi_{w_i}}(s)$ and $A^\beta((s, i), a) = A^{\pi_{w_i}}(s, a)$ for every (s, i) , where α_i is the batch fraction of version i . Consequently $J_{\text{mix}} = \sum_i \alpha_i J(\pi_{w_i})$.*

Since the index never changes, every future rollout from (s, i) is generated by the same π_{w_i} , so the augmented-state value and advantage reduce to those of π_{w_i} ; a proof is given in Appendix E.2.

Theorem 1 (Monotonic improvement, trajectory-level mixing) *Let the batch be a trajectory-level mixture ($q = \mathbf{1}\{i' = i\}$) with version fractions $\{\alpha_i\}$. Then, under the standard trust-region*

regularity conditions of Schulman et al. [27], Achiam et al. [1],

$$J(\pi_\theta) - \sum_i \alpha_i J(\pi_{w_i}) \geq \sum_i \alpha_i L_{\pi_{w_i}}(\pi_\theta) - \frac{2\gamma \max_i C_{\pi_\theta, \pi_{w_i}}}{(1-\gamma)^2} \sum_i \alpha_i \mathbb{E}_{s \sim d_{\pi_{w_i}}} [D_{\text{TV}}(\pi_\theta, \pi_{w_i}; s)]. \quad (5)$$

Equation (5) adapts the multi-version sample-reuse improvement bound of Queeney et al. [25] to the mixed-behavior setting. Unlike that bound, which anchors the advantage at the current policy and measures improvement relative to it, our bound anchors at the per-version advantage $A^{\pi_{w_i}}$ and measures improvement relative to the mixture baseline $\sum_i \alpha_i J(\pi_{w_i})$; the proof (via the performance-difference lemma [18] and the average-TV bound of Achiam et al. [1]) is given in Appendix E.3. The key structural point, supplied by Lemma 1, is that the surrogate uses each trajectory’s *own* behavior version π_{w_i} together with its *own* advantage $A^{\pi_{w_i}}$. Consequently the per-trajectory importance ratio π_θ / π_{w_i} —exactly the ratio $r_{i,t}$ in DORA’s objective—is a well-defined single-policy ratio, and the bound holds *without any cross-version correction*.

D.3 Advantage-Substitution Bias under Step/Segment-Level Mixing

The reduction of Lemma 1 is exactly what fails once trajectories switch versions mid-generation. This is the additive observation of this appendix.

Corollary 1 (Single-version generation removes advantage-substitution bias) *Under step/segment-level mixing ($\sigma > 0$), the augmented-state advantage $A^\beta((s, i), a)$ depends on the future index evolution induced by q and does not in general equal any single-version advantage $A^{\pi_{w_i}}$. Estimating the surrogate with $A^{\pi_{w_i}}$ in place of A^β therefore incurs an advantage-substitution bias*

$$\varepsilon_{\text{sub}}(i) := \left| \mathbb{E}_{s \sim d_{\pi_{w_i}}, a \sim \pi_{w_i}(\cdot|s)} [A^\beta((s, i), a) - A^{\pi_{w_i}}(s, a)] \right|, \quad (6)$$

which is nonzero for $\sigma > 0$ (whenever the switched-to version differs) and vanishes as $\sigma \rightarrow 0$. This bias enters the improvement guarantee directly. Writing $\hat{L}(\pi_\theta)$ for the surrogate actually optimized with the single-version advantage $A^{\pi_{w_i}}$, and $L^\beta(\pi_\theta) := \frac{1}{1-\gamma} \mathbb{E}_{(s,i) \sim d_\beta, a \sim \pi_{w_i}} [\frac{\pi_\theta(a|s)}{\pi_{w_i}(a|s)} A^\beta((s, i), a)]$ for the surrogate required by the augmented-state bound, a bounded importance ratio $\pi_\theta / \pi_{w_i} \leq M_\rho$ gives $|L^\beta(\pi_\theta) - \hat{L}(\pi_\theta)| \leq \frac{M_\rho}{1-\gamma} \mathbb{E}_i[\varepsilon_{\text{sub}}(i)]$. Substituting into the augmented-state improvement bound yields, for any σ ,

$$\begin{aligned} J(\pi_\theta) - J_{\text{mix}} &\geq \hat{L}(\pi_\theta) - \frac{2\gamma C_{\pi_\theta, \beta}}{(1-\gamma)^2} \mathbb{E}_{(s,i) \sim d_\beta} [D_{\text{TV}}(\pi_\theta, \pi_{w_i}; s)] \\ &\quad - \underbrace{\frac{M_\rho}{1-\gamma} \mathbb{E}_i[\varepsilon_{\text{sub}}(i)]}_{\text{substitution penalty}}. \end{aligned} \quad (7)$$

Trajectory-level mixing ($\sigma = 0$, DORA) is the unique regime in which $\varepsilon_{\text{sub}} \equiv 0$: the substitution penalty vanishes, the reduction of Lemma 1 holds exactly, and the standard GRPO objective directly realizes the bound’s surrogate without bias.

$\hat{L}$ is a best-case proxy. We take $\hat{L}$ to use the clean single-version advantage $A^{\pi_{w_i}}$; this is a *best-case* proxy for partial rollout, which in practice cannot even access $A^{\pi_{w_i}}$ exactly—its critic is trained on version-mixed data, or its group-relative advantage is shared across a version-switched trajectory. The gap ε_{sub} is therefore a *lower bound* on the true substitution error; even in this idealized case it is nonzero for $\sigma > 0$, whereas it vanishes identically for DORA ($\sigma = 0$).

D.4 Discussion: Consequences for DORA

Theorem 1 and Corollary 1 together give a policy-improvement reading of DORA’s single-policy-per-trajectory design along three axes, each mapping to a claim in the main text.

No algorithmic correction (Section 1, “without algorithmic compromises”). By Corollary 1, DORA’s $\sigma = 0$ makes the advantage reduction exact, so its surrogate is realized by the unmodified GRPO objective. Partial rollout’s $\sigma > 0$ breaks the reduction, which is why concrete systems must compensate with decoupled objectives [8] or gradient masking [36]. DORA’s freedom from such corrections is thus a direct consequence of trajectory-level mixing, not an engineering simplification.

Well-defined importance ratio (Section 3). Because each DORA trajectory corresponds to a single behavior version π_{w_i} , its sequence-level ratio $\prod_t \pi_\theta(a_t | \cdot) / \pi_{w_i}(a_t | \cdot)$ is a well-defined single-policy ratio. A trajectory stitched from multiple versions corresponds to no single behavior policy, so its sequence-level ratio loses this interpretation at segment boundaries—the algorithmic root of the corrections above.

Staleness cost and its control (Section 3, staleness bound K). The total-variation term in (5) grows with the deviation $D_{TV}(\pi_\theta, \pi_{w_i}; s)$ between the target policy and each behavior version. DORA controls this term on the sampling side: the sliding window enforces a deterministic staleness bound $v(\theta) - v(w_i) \leq K$ (Section 3), so the summands with the largest deviation are excluded by construction. This is the improvement-side counterpart of the convergence-throughput role of K discussed in the main text and its limitation (manual configuration, reliance on clipping). We note the trade-off honestly: step/segment-level mixing can reduce this deviation faster, since switching continually refreshes trajectories toward the latest version, whereas DORA holds long-tail trajectories at their dispatch version and instead bounds the deviation through K . The two paradigms are therefore not uniformly ordered; DORA trades a higher per-trajectory staleness ceiling—bounded by K —for an exact advantage reduction (Corollary 1), a well-defined single-policy ratio, and the KV-Cache equivalence that enables zero-re-prefill migration (Section 3).

E Proofs for the Policy-Improvement Analysis

This appendix collects the proofs for the results of Appendix D. All arguments are carried out on the *augmented* MDP, so that the mixed behavior policy is treated as a single Markov policy throughout; the per-version form (for DORA) is then recovered through Lemma 1. We first record the regularity conditions and the performance-difference identity (Appendix E.1), then prove Lemma 1 (Appendix E.2), Theorem 1 (Appendix E.3), and Corollary 1 (Appendix E.4).

E.1 Regularity Conditions and the Performance-Difference Identity

We work in the token-level MDP of Appendix D.1, with augmented state space $\tilde{\mathcal{S}} = \mathcal{S} \times \mathcal{I}$, mixed behavior policy $\beta(a | s, i) := \pi_{w_i}(a | s)$, and index-transition kernel $q(i' | i)$. The augmented MDP inherits the reward and environment transition of the base MDP, while the index evolves according to q independently of the action. Its initial state is drawn as $(s_0, i_0) \sim \rho_0 \times \alpha$, and the target policy π_θ is lifted to the augmented space by ignoring the index, $\pi_\theta(a | s, i) := \pi_\theta(a | s)$. Since the augmented transition is the product of the base transition and the index kernel q , it is a valid Markov kernel; consequently the state-distribution and performance-difference results below, established for general MDPs by Kakade and Langford [18] and Achiam et al. [1], apply on the augmented MDP verbatim.

Horizon convention. An LLM response is a finite length- H trajectory, whereas the bounds below are stated in the infinite-horizon discounted form for continuity with the classical policy-improvement literature [18, 27, 1]. The same decomposition holds in the finite-horizon setting, with the factor $1/(1 - \gamma)$ replaced by a horizon-dependent constant; we adopt the discounted notation throughout and do not track this substitution.

We assume:

- **Discounting.** $\gamma \in (0, 1)$, so every Bellman operator below is a contraction and the value functions are well defined.
- **Bounded advantage.** There exists $A_{\max} < \infty$ with $|A^\pi(s, a)| \leq A_{\max}$ for the policies considered.
- **Common support.** For every behavior version π_{w_i} entering the objective, $\pi_\theta(a | s) > 0 \Rightarrow \pi_{w_i}(a | s) > 0$, so the ratio π_θ / π_{w_i} is well defined.

- **Bounded ratio.** $\pi_\theta(a \mid s)/\pi_{w_i}(a \mid s) \leq M_\rho$, as enforced in practice by the clipping mechanism of PPO/GRPO [28, 30].

We use the performance-difference lemma [18] on the augmented MDP: for any augmented-space policies π, π' ,

$$J(\pi) - J(\pi') = \frac{1}{1-\gamma} \mathbb{E}_{(s,i) \sim d_\pi} \left[\mathbb{E}_{a \sim \pi(\cdot \mid s,i)} [A^{\pi'}((s,i), a)] \right], \quad (8)$$

and its consequence, the single-behavior trust-region improvement bound [27, 1]: for a behavior policy μ whose support covers π ,

$$J(\pi) - J(\mu) \geq \frac{1}{1-\gamma} \mathbb{E}_{(s,i) \sim d_\mu, a \sim \mu(\cdot \mid s,i)} \left[\frac{\pi(a \mid s,i)}{\mu(a \mid s,i)} A^\mu((s,i), a) \right] - \frac{2\gamma C_{\pi,\mu}}{(1-\gamma)^2} \mathbb{E}_{(s,i) \sim d_\mu} [D_{\text{TV}}(\pi, \mu; (s,i))], \quad (9)$$

with $C_{\pi,\mu} := \max_{(s,i)} |\mathbb{E}_{a \sim \pi(\cdot \mid s,i)} [A^\mu((s,i), a)]|$. The bound (9) follows from (8) by replacing the on-policy visitation d_π with the behavior visitation d_μ and controlling the induced distribution mismatch via the average total-variation bound of Achiam et al. [1], which holds on the augmented MDP by the remark above.

E.2 Proof of Lemma 1 (Advantage Reduction)

Proof 1 Assume trajectory-level mixing, $q(i' \mid i) = \mathbf{1}\{i' = i\}$. Under this kernel the index component is invariant: starting from (s, i) , every subsequent augmented state has the form (s', i) with the same i . Hence along any trajectory initialized at (s, i) the behavior policy $\beta(\cdot \mid \cdot, i) = \pi_{w_i}(\cdot \mid \cdot)$ acts as the fixed base-MDP policy π_{w_i} , and the augmented transition reduces to the base transition under π_{w_i} . Since $\gamma < 1$, the augmented Bellman equation has a unique solution, which therefore coincides with the base-MDP value of π_{w_i} :

$$V^\beta(s, i) = \mathbb{E} \left[\sum_{t \geq 0} \gamma^t r(s_t, a_t) \mid s_0 = s, \text{ index frozen at } i, a_t \sim \pi_{w_i} \right] = V^{\pi_{w_i}}(s),$$

and likewise $Q^\beta((s, i), a) = Q^{\pi_{w_i}}(s, a)$. Subtracting gives $A^\beta((s, i), a) = A^{\pi_{w_i}}(s, a)$ for every (s, i) ; this is a property of the value functions and does not depend on the visitation distribution. Finally, because the index is drawn once as $i_0 \sim \alpha$ and then held fixed while the base state evolves under π_{w_i} from $s_0 \sim \rho_0$, the augmented visitation factorizes as $d_\beta(s, i) = \alpha_i d_{\pi_{w_i}}(s)$, where $d_{\pi_{w_i}}$ is the base-MDP visitation of π_{w_i} started from ρ_0 . Integrating the reward identity over d_β then gives $J_{\text{mix}} = J(\beta) = \sum_i \alpha_i J(\pi_{w_i})$.

E.3 Proof of Theorem 1 (Monotonic Improvement)

Proof 2 We apply the trust-region bound (9) on the augmented MDP with target $\pi = \pi_\theta$ (lifted) and behavior $\mu = \beta$; this step uses only that β is a Markov policy on the augmented MDP and holds for any kernel q . Since π_θ ignores the index and $\beta(\cdot \mid s, i) = \pi_{w_i}(\cdot \mid s)$, the two policies at a fixed (s, i) are $\pi_\theta(\cdot \mid s)$ and $\pi_{w_i}(\cdot \mid s)$, so both the ratio and the total variation reduce to base-MDP quantities,

$$\frac{\pi_\theta(a \mid s,i)}{\beta(a \mid s,i)} = \frac{\pi_\theta(a \mid s)}{\pi_{w_i}(a \mid s)}, \quad D_{\text{TV}}(\pi_\theta, \beta; (s, i)) = D_{\text{TV}}(\pi_\theta, \pi_{w_i}; s).$$

With $J(\beta) = J_{\text{mix}}$, (9) becomes the augmented-state bound

$$J(\pi_\theta) - J_{\text{mix}} \geq \frac{1}{1-\gamma} \mathbb{E}_{(s,i) \sim d_\beta, a \sim \pi_{w_i}} \left[\frac{\pi_\theta(a \mid s)}{\pi_{w_i}(a \mid s)} A^\beta((s, i), a) \right] - \frac{2\gamma C_{\pi_\theta, \beta}}{(1-\gamma)^2} \mathbb{E}_{(s,i) \sim d_\beta} [D_{\text{TV}}(\pi_\theta, \pi_{w_i}; s)], \quad (10)$$

which holds for any kernel q . We now specialize to trajectory-level mixing and expand via Lemma 1. The factorization $d_\beta(s, i) = \alpha_i d_{\pi_{w_i}}(s)$ turns every augmented expectation $\mathbb{E}_{(s,i) \sim d_\beta} [\cdot]$ into $\sum_i \alpha_i \mathbb{E}_{s \sim d_{\pi_{w_i}}} [\cdot]$, and the reduction $A^\beta((s, i), a) = A^{\pi_{w_i}}(s, a)$ replaces the augmented advantage by the per-version advantage. The surrogate term becomes

$$\frac{1}{1-\gamma} \sum_i \alpha_i \mathbb{E}_{s \sim d_{\pi_{w_i}}, a \sim \pi_{w_i}} \left[\frac{\pi_\theta(a \mid s)}{\pi_{w_i}(a \mid s)} A^{\pi_{w_i}}(s, a) \right] = \sum_i \alpha_i L_{\pi_{w_i}}(\pi_\theta),$$

with $L_{\pi_{w_i}}$ as in (4), and the penalty term becomes $\frac{2\gamma C_{\pi_{\theta}, \beta}}{(1-\gamma)^2} \sum_i \alpha_i \mathbb{E}_{s \sim d_{\pi_{w_i}}} [D_{\text{TV}}(\pi_{\theta}, \pi_{w_i}; s)]$. Finally, since the maximum over the product space factorizes and $A^{\beta}((s, i), a) = A^{\pi_{w_i}}(s, a)$,

$$C_{\pi_{\theta}, \beta} = \max_{(s, i)} |\mathbb{E}_{a \sim \pi_{\theta}} [A^{\beta}((s, i), a)]| = \max_i \max_s |\mathbb{E}_{a \sim \pi_{\theta}} [A^{\pi_{w_i}}(s, a)]| = \max_i C_{\pi_{\theta}, \pi_{w_i}}.$$

Substituting these three identities into (10) yields (5).

E.4 Proof of Corollary 1 (Advantage-Substitution Bias)

Throughout this proof we work under the stylized kernel $q(i' | i) = (1 - \sigma)\mathbf{1}\{i' = i\} + \sigma \kappa(i' | i)$ of Appendix D.1, and $A^{\pi_{w_i}}$ denotes the advantage of the current-segment version, i.e., the version indexed by the augmented state (s, i) .

Proof 3 When $\sigma = 0$ the kernel is the identity transition and Lemma 1 gives $A^{\beta}((s, i), a) = A^{\pi_{w_i}}(s, a)$ pointwise, hence $\varepsilon_{\text{sub}}(i) = 0$. When $\sigma > 0$, the augmented value obeys

$$V^{\beta}(s, i) = \mathbb{E}_{a \sim \pi_{w_i}} \left[r(s, a) + \gamma \mathbb{E}_{s'} [(1 - \sigma)V^{\beta}(s', i) + \sigma \mathbb{E}_{i' \sim \kappa(\cdot | i)} V^{\beta}(s', i')] \right], \quad (11)$$

which differs from the $\sigma = 0$ recursion (whose solution is $V^{\pi_{w_i}}$) through the σ -weighted term: with probability σ the continuation is evaluated under a version $i' \sim \kappa(\cdot | i)$. Whenever $\pi_{w_{i'}} \neq \pi_{w_i}$ we have $V^{\beta}(\cdot, i') \neq V^{\pi_{w_i}}(\cdot)$, so the perturbing term is nonzero and, barring exact cancellation in expectation, $A^{\beta}((s, i), a) \neq A^{\pi_{w_i}}(s, a)$, giving $\varepsilon_{\text{sub}}(i) > 0$. The σ -weighting of the perturbing term in (11) makes the deviation grow with σ and with the inter-version value gap.

Entry into the improvement bound. Let $\hat{L}(\pi_{\theta})$ be the surrogate optimized with the single-version advantage $A^{\pi_{w_i}}$, and $L^{\beta}(\pi_{\theta})$ the surrogate in the augmented bound (10) with the true mixed advantage A^{β} ; both are evaluated on the same sampling distribution d_{β} . Their difference is

$$L^{\beta}(\pi_{\theta}) - \hat{L}(\pi_{\theta}) = \frac{1}{1 - \gamma} \mathbb{E}_{(s, i) \sim d_{\beta}, a \sim \pi_{w_i}} \left[\frac{\pi_{\theta}(a | s)}{\pi_{w_i}(a | s)} (A^{\beta}((s, i), a) - A^{\pi_{w_i}}(s, a)) \right].$$

The total-variation penalty in (10) contains no advantage and is unchanged by the substitution, so it cancels in $L^{\beta} - \hat{L}$; only the surrogate is affected. Bounding the ratio by M_{ρ} and applying the definition (6),

$$|L^{\beta}(\pi_{\theta}) - \hat{L}(\pi_{\theta})| \leq \frac{M_{\rho}}{1 - \gamma} \mathbb{E}_i [\varepsilon_{\text{sub}}(i)].$$

Writing $\hat{L} = L^{\beta} - (L^{\beta} - \hat{L})$ in the augmented bound (10) and lower-bounding $L^{\beta} - \hat{L} \geq -|L^{\beta} - \hat{L}|$ moves this gap to the right-hand side as an additive penalty, yielding (7). At $\sigma = 0$ the penalty vanishes and the standard GRPO objective realizes the bound's surrogate exactly.

F A Control-Variate View of Baseline Choice in DORA vs. Partial Rollout

Complementary to the policy-improvement analysis of Appendix D, which examined how single-version generation affects the monotonic-improvement guarantee, this appendix turns to its effect on the variance of the policy-gradient estimator. DORA and partial-rollout methods [36, 41, 6, 8] differ along a second, subtler algorithmic dimension: the choice of constant baseline used in advantage estimation for group-relative policy optimization. We examine this dimension through the classical control-variate framework for policy gradients [40, 10]. Our goal is again modest: to provide a clean lens through which two baseline choices can be located and compared, and to describe the structure of the variance gap between them within a stylized abstraction.

Scope and abstraction. The analysis below is a stylized comparison of two constant baselines under a common sampling abstraction, not a faithful model of either system's sampling process. We reuse the single-version notation of Appendix D: π_{θ} is the target policy and $\{\pi_{w_k}\}$ the lagged versions with schedule $\{\alpha_k\}$. Concretely, we adopt the following single-version sampling abstraction—the $\sigma = 0$ regime of Appendix D: each sample is a triple (k, s, a) with $k \sim \alpha$, $s \sim \rho_0$, and $a \sim \pi_{w_k}(\cdot | s)$, so that the response is generated end-to-end under a single policy version π_{w_k} . This abstraction is faithful to DORA and serves as a stylized stand-in for version-mixing methods, whose stitched

trajectories it does not model in full. Both estimators studied below are defined with respect to this abstraction; they share the same sampling distribution and the same importance-weighted score $S_k(s, a) = (\pi_\theta / \pi_{w_k}) \nabla_\theta \log \pi_\theta(a | s)$, and differ only in the constant baseline subtracted from the reward.

This abstraction is faithful to DORA’s actual sampling process, since DORA generates each trajectory end-to-end under a single policy version. It is *not* faithful to partial-rollout methods, whose trajectories are stitched from segments produced under multiple policy versions within the staleness window.² We make no claim that the analysis below captures partial rollout’s true sampling variance. Instead, we use the version-specific baseline μ_k as the formal counterpart of DORA’s design (which can name the version of any trajectory) and the schedule-averaged baseline $\bar{\mu} = \sum_k p_k \mu_k$ as the natural representative of the partial-rollout family within this abstraction (which by design does not single out any one version of a trajectory). The resulting comparison isolates the dimension of *baseline choice* from orthogonal mechanisms used by concrete partial-rollout systems—decoupled objectives, importance corrections across stitched segments, gradient masking, and so on—which lie outside our scope. Following GRPO [30] and the asynchronous RL systems built on it [8, 36, 41], we restrict attention to constant (state-independent) baselines; learned state-dependent baselines (as in PPO) are outside our scope. All proofs are deferred to Appendix G.

F.1 Setup and Notation

We adopt the sequence-level contextual bandit formulation [30, 2]: states $s \sim \rho_0$ are prompts drawn from a static distribution, actions $a \sim \pi(\cdot | s)$ are full responses, and rewards $R(s, a) \in [0, R_{\max}]$ are verifiable correctness signals. Let π_θ denote the target policy and $\{\pi_{w_k}\}_{k=1}^K$ the lagged policies with schedule $\{p_k\}$. Define

$$\mu_k = \mathbb{E}_{s \sim \rho_0, a \sim \pi_{w_k}}[R(s, a)], \quad \bar{\mu} = \sum_k p_k \mu_k, \quad (12)$$

and the importance-weighted score

$$S_k(s, a) = \frac{\pi_\theta(a | s)}{\pi_{w_k}(a | s)} \nabla_\theta \log \pi_\theta(a | s), \quad (13)$$

where ∇_θ denotes the gradient with respect to the policy parameters θ . Note that $\bar{\mu}$ is a deterministic constant, whereas μ_k is a random variable depending on the sampled version index k . We define three local second-order quantities, all conditional on a fixed version k with $(s, a) \sim \rho_0 \times \pi_{w_k}$:

$$B_k = \mathbb{E}[\|S_k\|^2 | k], \quad C_k = \mathbb{E}[(R - \mu_k)\|S_k\|^2 | k], \quad V_k = \mathbb{E}[(R - \mu_k)^2\|S_k\|^2 | k]. \quad (14)$$

Since $\mu_k = \mathbb{E}[R | k]$ and $B_k = \mathbb{E}[\|S_k\|^2 | k]$, the quantity C_k admits the equivalent interpretation

$$C_k = \text{Cov}_{(s,a)|k}(R, \|S_k\|^2), \quad (15)$$

i.e., the within-version covariance between reward and squared score. Throughout, $\text{Var}(\cdot)$ denotes $\mathbb{E}[\|\cdot - \mathbb{E}[\cdot]\|^2] = \text{tr}(\text{Cov}(\cdot))$ for vector-valued random variables, and $\text{Var}_p(\cdot)$, $\text{Cov}_p(\cdot, \cdot)$ denote variance and covariance under the schedule distribution p over versions.

The two estimators we compare are

$$\hat{g}_{\mu_k} = (R - \mu_k) S_k, \quad \hat{g}_{\bar{\mu}} = (R - \bar{\mu}) S_k, \quad (16)$$

both with $k \sim p$ and $(s, a) \sim \rho_0 \times \pi_{w_k}$ under the single-version sampling abstraction. Within this abstraction, $\hat{g}_{\mu_k}$ corresponds to DORA’s design (which can identify the version of every trajectory), and $\hat{g}_{\bar{\mu}}$ corresponds to the partial-rollout family within the abstraction (which by design does not single out one version per trajectory).³ We analyze single-sample variance throughout; batch-averaged variance scales as $1/G$ where G is the batch size, so the comparative results extend directly to the batched case. Under standard importance-sampling regularity (Appendix G.1), both estimators are unbiased for $\nabla_\theta J(\pi_\theta)$.

²For partial-rollout systems that apply token-level importance correction, the stitched and single-version sampling distributions agree in expectation, which provides a partial justification for the abstraction; their variance behavior nonetheless differs in general.

³We re-emphasize that $\hat{g}_{\bar{\mu}}$ is a stylized stand-in: concrete partial-rollout systems include additional mechanisms—decoupled objectives, importance corrections across stitched segments, gradient masking—that are not represented by $\hat{g}_{\bar{\mu}}$. The variance comparison below reflects only the baseline-choice dimension.

F.2 Optimal Constant Baseline

We first characterize the variance-minimizing constant baseline, which serves as the reference point against which both μ_k and $\bar{\mu}$ are approximations.

Lemma 2 (Optimal Constant Control-Variate Baseline) *For any fixed version k with $B_k > 0$ and $\mathbb{E}[R^2 \|S_k\|^2 \mid k] < \infty$, the constant baseline $b \in \mathbb{R}$ minimizing $\text{Var}((R - b)S_k \mid k)$ is unique and given by*

$$b_k^* = \frac{\mathbb{E}[R \|S_k\|^2 \mid k]}{\mathbb{E}[\|S_k\|^2 \mid k]} = \mu_k + \frac{C_k}{B_k}. \quad (17)$$

Equation (17) expresses b_k^* as the sum of two additive terms: the version-mean reward μ_k , and a correction term C_k/B_k capturing the within-version reward–gradient coupling. The second term requires per-sample gradient norm estimates and is generally impractical to compute in sequence generation settings—a difficulty noted explicitly by Hao et al. [13], who similarly invoke simplifying assumptions to obtain a tractable form of the optimal baseline. Both baselines we consider drop this correction term:

- μ_k retains version-level reward information; within our abstraction, this is the baseline available to a method that can identify the version of every trajectory. For a given k , $\mu_k = b_k^*$ when $C_k/B_k = 0$.
- $\bar{\mu} = \sum_k p_k \mu_k$ aggregates across versions; within our abstraction, this is the baseline available to a method that does not. The condition $\bar{\mu} = b_k^*$ for all k requires both $C_k/B_k = 0$ for all k and μ_k constant in k .

The next subsection quantifies the variance gap between these two choices.

F.3 Exact Variance Decomposition

The variance behavior of the two baselines is characterized by the following exact identity.

Theorem 2 (Variance Gap Decomposition) *Under the single-version sampling abstraction and the regularity conditions of Appendix G.1, the variance gap between the two estimators decomposes as*

$$\Delta \text{Var} := \text{Var}(\hat{g}_{\bar{\mu}}) - \text{Var}(\hat{g}_{\mu_k}) = \underbrace{\mathbb{E}_{k \sim p}[B_k (\mu_k - \bar{\mu})^2]}_{\text{drift term}} + \underbrace{2 \text{Cov}_p(\mu_k, C_k)}_{\text{coupling term}}. \quad (18)$$

Reading the decomposition. The decomposition isolates two distinct sources of variance difference. The drift term is non-negative by construction; the coupling term has indeterminate sign, so ΔVar itself can in principle have either sign.

- The **drift term** $\mathbb{E}_{k \sim p}[B_k (\mu_k - \bar{\mu})^2]$ is the cross-version pass-rate variance weighted by gradient energy. It quantifies the cost, within the abstraction, of replacing the version-specific mean μ_k with a single cross-version aggregate $\bar{\mu}$: it grows whenever pass rates differ across versions, and vanishes when they coincide.
- The **coupling term** $2 \text{Cov}_p(\mu_k, C_k)$ captures whether higher-pass-rate versions also tend to exhibit larger reward–gradient covariance (positive sign, amplifying the gap) or the opposite (negative sign, partially offsetting it). Its sign and magnitude depend on the joint distribution of (μ_k, C_k) across versions.

F.4 Structural Observations

Theorem 2 reduces ΔVar to two terms with distinct structural roles. We record a Cauchy–Schwarz bound on the coupling term and note what it does and does not imply.

Cauchy–Schwarz upper bound on the coupling term. Applying Cauchy–Schwarz under the schedule distribution p ,

$$|2 \text{Cov}_p(\mu_k, C_k)| \leq 2 \sqrt{\text{Var}_p(\mu_k) \text{Var}_p(C_k)}. \quad (19)$$

Under the bounded-reward and bounded-score conditions of Appendix G.1, $|C_k| \leq 2R_{\max}M$ for every k , so $\text{Var}_p(C_k) < \infty$ and the inequality is well-defined. The bound expresses that the coupling term is constrained by the cross-version variability of μ_k and of the within-version reward–gradient covariance C_k ; it does not by itself imply any direction for ΔVar .

What this analysis does and does not establish. We do not claim that $\hat{g}_{\mu_k}$ has uniformly smaller variance than $\hat{g}_{\bar{\mu}}$, even within the abstraction. Theorem 2 is explicit that ΔVar can have either sign, and the magnitudes of both terms depend on the training regime in ways we do not attempt to characterize. What the analysis does establish, within the single-version sampling abstraction, is: (i) the gap admits an exact decomposition into a non-negative drift term and a sign-indeterminate coupling term, and (ii) the coupling term is structurally bounded by (19). We do not extrapolate to claims about the variance of full partial-rollout systems, which involve additional mechanisms outside the abstraction.

F.5 Summary

This appendix has located two baseline choices within the control-variate framework, under a common single-version sampling abstraction. Both are tractable approximations to the optimal constant baseline $b_k^* = \mu_k + C_k/B_k$ that drop the impractical coupling term C_k/B_k . The version-specific baseline μ_k —available to methods, like DORA, that can identify the version of every trajectory—retains version-level reward information; the schedule-averaged baseline $\bar{\mu}$ —used here as the representative within the abstraction of methods that do not—aggregates across versions. Theorem 2 gives an exact decomposition of the variance gap into a non-negative drift term and a sign-indeterminate coupling term. The framework’s value is descriptive: it offers a structural lens on the algorithmic dimension of single-policy-per-trajectory generation, complementing the system-level benefit (zero-re-prefill migration).

G Proofs for the Control-Variate Analysis

This appendix provides full proofs for the results in Appendix F. We collect the regularity assumptions and unbiasedness identities in Appendix G.1, prove Lemma 2 in Appendix G.2, and prove Theorem 2 in Appendix G.3.

G.1 Regularity Assumptions and Unbiasedness

The analysis in Appendix F relies on the following standard regularity conditions:

- **Bounded reward:** $R(s, a) \in [0, R_{\max}]$ almost surely.
- **Bounded score:** For all versions k , $\|S_k(s, a)\|^2 \leq M$ almost surely under $(s, a) \sim \rho_0 \times \pi_{w_k}$. In practice, this is enforced by the importance-ratio clipping mechanism of PPO and GRPO [28, 30], which prevents the score from diverging. Combined with bounded reward, this implies B_k, C_k, V_k are all finite, with $|C_k| \leq 2R_{\max}M$ and $V_k \leq R_{\max}^2M$ for every k .
- **Non-degenerate gradient energy:** For all k , $B_k = \mathbb{E}[\|S_k\|^2 \mid k] \geq B_{\min} > 0$. This ensures that the target policy maintains a nontrivial gradient signal and prevents division-by-zero singularities in the derivation of the optimal baseline.
- **Importance-sampling regularity:** The standard support condition $\pi_\theta \ll \pi_{w_k}$ holds for all k . By direct change of measure,

$$\mathbb{E}[S_k(s, a) \mid k] = \mathbb{E}_{s \sim \rho_0} \left[\int \pi_\theta(a \mid s) \nabla_\theta \log \pi_\theta(a \mid s) da \right] = \mathbb{E}_{s \sim \rho_0} [\nabla_\theta 1] = 0, \quad (20)$$

using the identity $\int \pi_\theta \nabla_\theta \log \pi_\theta da = \nabla_\theta \int \pi_\theta da = \nabla_\theta 1 = 0$. Furthermore, by the same change of measure,

$$\mathbb{E}[R(s, a) S_k(s, a) \mid k] = \mathbb{E}_{(s, a) \sim \rho_0 \times \pi_\theta} [R(s, a) \nabla_\theta \log \pi_\theta(a \mid s)] = \nabla_\theta J(\pi_\theta), \quad (21)$$

which is independent of k .

Conditional mean of a generic baseline-shifted estimator. Conditional on a fixed version k , any constant b is deterministic. Combining (20) and (21) yields, for every constant $b \in \mathbb{R}$,

$$\mathbb{E}[(R - b) S_k \mid k] = \mathbb{E}[R S_k \mid k] - b \mathbb{E}[S_k \mid k] = \nabla_\theta J(\pi_\theta) - b \cdot 0 = \nabla_\theta J(\pi_\theta). \quad (22)$$

Specializing $b = \mu_k$ and $b = \bar{\mu}$ gives $\mathbb{E}[\hat{g}_{\mu_k} \mid k] = \mathbb{E}[\hat{g}_{\bar{\mu}} \mid k] = \nabla_\theta J(\pi_\theta)$. Since the conditional mean is k -independent, taking expectation over $k \sim p$ preserves it: both estimators are unbiased for $\nabla_\theta J(\pi_\theta)$.

G.2 Proof of Lemma 2

We work conditional on a fixed version k throughout this proof; all expectations are over $(s, a) \sim \rho_0 \times \pi_{w_k}$. The proof requires only the conditions stated in Lemma 2—namely $B_k > 0$ and $\mathbb{E}[R^2 \|S_k\|^2 \mid k] < \infty$ —together with the importance-sampling regularity of Appendix G.1 (used to establish (22)).

By definition of the trace covariance,

$$\text{Var}((R - b) S_k \mid k) = \mathbb{E}[\|(R - b) S_k\|^2 \mid k] - \|\mathbb{E}[(R - b) S_k \mid k]\|^2. \quad (23)$$

Since $R - b$ is a scalar, $\|(R - b) S_k\|^2 = (R - b)^2 \|S_k\|^2$. By (22), $\mathbb{E}[(R - b) S_k \mid k] = \nabla_\theta J(\pi_\theta)$ for every constant b , so the second term in (23) is independent of b . Minimizing $\text{Var}((R - b) S_k \mid k)$ over $b \in \mathbb{R}$ is therefore equivalent to minimizing the scalar function

$$f(b) := \mathbb{E}[(R - b)^2 \|S_k\|^2 \mid k] = \mathbb{E}[R^2 \|S_k\|^2 \mid k] - 2b \mathbb{E}[R \|S_k\|^2 \mid k] + b^2 B_k.$$

This is a strictly convex quadratic in b (since $f''(b) = 2B_k > 0$), so the unique minimizer is given by the first-order condition $f'(b) = 0$:

$$-2 \mathbb{E}[R \|S_k\|^2 \mid k] + 2b B_k = 0 \implies b_k^* = \frac{\mathbb{E}[R \|S_k\|^2 \mid k]}{B_k}.$$

To connect this to μ_k , add and subtract μ_k in the numerator:

$$b_k^* = \frac{\mathbb{E}[(R - \mu_k) + \mu_k] \|S_k\|^2 \mid k]}{B_k} = \frac{\mu_k B_k + C_k}{B_k} = \mu_k + \frac{C_k}{B_k},$$

by the definitions of B_k and C_k in (14).

G.3 Proof of Theorem 2

Step 1: Reduction to conditional variances via the law of total variance. For each estimator $\hat{g} \in \{\hat{g}_{\mu_k}, \hat{g}_{\bar{\mu}}\}$, the matrix law of total covariance states

$$\text{Cov}(\hat{g}) = \mathbb{E}_{k \sim p}[\text{Cov}(\hat{g} \mid k)] + \text{Cov}_{k \sim p}(\mathbb{E}[\hat{g} \mid k]).$$

Taking the trace of both sides and using linearity of trace yields the analogous identity for the trace-covariance scalar $\text{Var}(\cdot)$ defined in Section F.1:

$$\text{Var}(\hat{g}) = \mathbb{E}_{k \sim p}[\text{Var}(\hat{g} \mid k)] + \text{Var}_{k \sim p}(\mathbb{E}[\hat{g} \mid k]). \quad (24)$$

By Appendix G.1, $\mathbb{E}[\hat{g} \mid k] = \nabla_\theta J(\pi_\theta)$ for both estimators, which is k -independent. The second term in (24) therefore vanishes, yielding $\text{Var}(\hat{g}) = \mathbb{E}_{k \sim p}[\text{Var}(\hat{g} \mid k)]$.

Step 2: Conditional variance for a generic constant baseline. Fix k and consider the estimator $\hat{g}_b = (R - b) S_k$ with b a constant. Combining (23) with $\mathbb{E}[(R - b) S_k \mid k] = \nabla_\theta J(\pi_\theta)$ from (22),

$$\text{Var}(\hat{g}_b \mid k) = \mathbb{E}[(R - b)^2 \|S_k\|^2 \mid k] - \|\nabla_\theta J(\pi_\theta)\|^2, \quad (25)$$

where the second term is independent of b .

Step 3: Algebraic decomposition of the second moment. For $b = \bar{\mu}$, complete the square around μ_k :

$$(R - \bar{\mu})^2 = (R - \mu_k)^2 + (\mu_k - \bar{\mu})^2 + 2(R - \mu_k)(\mu_k - \bar{\mu}).$$

Multiplying by $\|S_k\|^2$ and taking conditional expectation, with $(\mu_k - \bar{\mu})$ a deterministic constant given k ,

$$\begin{aligned} \mathbb{E}[(R - \bar{\mu})^2 \|S_k\|^2 \mid k] &= \underbrace{\mathbb{E}[(R - \mu_k)^2 \|S_k\|^2 \mid k]}_{=V_k} + (\mu_k - \bar{\mu})^2 \underbrace{\mathbb{E}[\|S_k\|^2 \mid k]}_{=B_k} \\ &\quad + 2(\mu_k - \bar{\mu}) \underbrace{\mathbb{E}[(R - \mu_k) \|S_k\|^2 \mid k]}_{=C_k} \\ &= V_k + B_k(\mu_k - \bar{\mu})^2 + 2(\mu_k - \bar{\mu}) C_k. \end{aligned} \quad (26)$$

For $b = \mu_k$, no completion is needed:

$$\mathbb{E}[(R - \mu_k)^2 \|S_k\|^2 \mid k] = V_k. \quad (27)$$

Step 4: Aggregating over k and subtracting. Combining Steps 1–3,

$$\begin{aligned} \text{Var}(\hat{g}_{\bar{\mu}}) &= \mathbb{E}_{k \sim p}[V_k + B_k(\mu_k - \bar{\mu})^2 + 2(\mu_k - \bar{\mu}) C_k] - \|\nabla_{\theta} J(\pi_{\theta})\|^2, \\ \text{Var}(\hat{g}_{\mu_k}) &= \mathbb{E}_{k \sim p}[V_k] - \|\nabla_{\theta} J(\pi_{\theta})\|^2. \end{aligned}$$

Subtracting, the $\mathbb{E}_p[V_k]$ terms and the $\|\nabla_{\theta} J(\pi_{\theta})\|^2$ terms cancel exactly:

$$\Delta \text{Var} = \mathbb{E}_{k \sim p}[B_k(\mu_k - \bar{\mu})^2] + 2 \mathbb{E}_{k \sim p}[(\mu_k - \bar{\mu}) C_k]. \quad (28)$$

Step 5: Identifying the cross term as a covariance. Since $\mathbb{E}_{k \sim p}[\mu_k] = \bar{\mu}$, we have $\mathbb{E}_p[\mu_k - \bar{\mu}] = 0$. Therefore,

$$\text{Cov}_p(\mu_k, C_k) = \mathbb{E}_p[(\mu_k - \bar{\mu})(C_k - \mathbb{E}_p[C_k])] = \mathbb{E}_p[(\mu_k - \bar{\mu}) C_k] - \underbrace{\mathbb{E}_p[\mu_k - \bar{\mu}] \cdot \mathbb{E}_p[C_k]}_{=0} = \mathbb{E}_p[(\mu_k - \bar{\mu}) C_k].$$

Substituting into (28) yields the claimed identity:

$$\Delta \text{Var} = \mathbb{E}_{k \sim p}[B_k(\mu_k - \bar{\mu})^2] + 2 \text{Cov}_p(\mu_k, C_k).$$

H Social impacts

By detailing our methodology and experimental results in production environments, we aim to advance both research and industrial practices in the field of large language model (LLM) training. Through improving training efficiency, DORA has the potential to meaningfully reduce the environmental footprint associated with training LLMs, which typically demand hundreds or even thousands of accelerators. Furthermore, our methodology has been validated on large-scale, non-CUDA mid-range accelerators, broadening its applicability beyond conventional hardware ecosystems. This not only benefits large organizations seeking hardware flexibility but also contributes to the democratization of LLM training by making it more accessible on less advanced accelerators.